

Tanuki: New Frameworks for (Concurrently Secure) Blind Signatures from Post-Quantum Groups Actions

Lucjan Hanzlik¹ , Yi-Fu Lai² , Marzio Mula³ ,
Eugenio Paracucchi¹, Daniel Slamanig³ , and Gang Tang⁴ ,

¹ CISP Helmholtz Center for Information Security

² Ruhr-Universität Bochum

³ Universität der Bundeswehr München

⁴ University of Birmingham

Abstract. Blind signatures are fundamental cryptographic primitives enabling privacy-preserving authentication and have seen renewed interest in the post-quantum literature. Existing efficient constructions predominantly rely on Fischlin’s generic paradigm instantiated over lattice assumptions, while blinding techniques for Σ -protocol-based blind signatures remain sparse beyond lattices. Moreover, achieving provable concurrent security under polynomially many sessions has been a long-standing open challenge for this approach in the post-quantum literature as evidenced by the recent attacks in EC’24 and PKC’24.

This work broadens the landscape of post-quantum blind signatures by introducing novel techniques and proposing four frameworks based on general cryptographic group actions, without requiring commutativity. Our constructions admit instantiations under diverse post-quantum assumptions, including CSIDH (isogeny-based), LESS (code-based, NIST round-two), and more. These frameworks offer flexible trade-offs in assumptions (from interactive one-more to the standard inversion problem) and key/signature sizes, and culminate in a construction that achieves security under polynomially many concurrent sessions. This enables the first efficient blind signatures from isogenies and codes with provable concurrent security with 4.5 and 64.7 KB respectively. We also outline several directions for optimization and further instantiations for future work.

1 Introduction

Blind signatures, introduced by Chaum [37], are a foundational cryptographic primitive that enables a signer to interactively issue signatures on messages chosen by the user without learning the content. This privacy-preserving building block underpins a wide range of applications, including e-cash, e-voting, anonymous credentials, and modern systems like Privacy Pass [48] and Tumblebit [66].

The core privacy feature of blind signatures is *blindness*, which prevents the signer from linking the issuance process to the final signature and learning meaningful information about the user’s message. Additionally, they ensure

unforgeability, meaning users cannot create valid signatures without the signer’s participation. Formally, this property is termed *one-more unforgeability*, where adversaries are challenged to produce $\ell + 1$ valid signatures after at most ℓ interactions with the signing oracle. Depending on whether the adversary can open multiple sessions simultaneously or sequentially, the security is categorized as *concurrent* or *sequential*, respectively.

Practical blind signatures have been proposed from various classical assumptions, including RSA [37], discrete logarithms [83,64,36], or pairings [29,60,63]. Unfortunately, the post-quantum landscape remains less favorable when it comes to diversity of assumptions as well as efficiency. Broadly speaking, there are two main approaches for constructing blind signatures. The first follows Fischlin’s generic template [59], which has proven particularly fruitful in the lattice-based setting. Recent works [3,82], culminating in that of Beullens et al. [22], have reduced the signature size to approximately 22 kilobytes by using a combination of the SIS, LWE, and NTRU assumptions. These efficiency gains stem largely from advances in lattice-based NIZK proof systems for structured languages [31,77]. Due to the lack of similarly efficient NIZKs for other post-quantum assumptions, it does not seem possible to replicate such improvements using Fischlin’s paradigm outside the setting. Using zk-SNARKs to build post-quantum signatures demands tens of thousands of gates over small fields or several hundred gates over large fields, resulting in expensive overhead in the application. For example, the state-of-the-art isogeny proof of knowledge requires approximately a thousand multiplication gates and proofs of at least several hundred kilobytes in size [42].

The second approach is to build on a sigma protocol (or an identification protocol) with a “nice” algebraic structure and to transform it into a blind signature via appropriate blinding of the interaction. Unfortunately, such algebraic structures or say the *blinding techniques* have been scarce in the post-quantum setting. For some time, the only known general method relied on the family of *linear hash functions* [64,65], which to date admit only lattice-based instantiations in the post-quantum literature. It took a decade to turn Lyubashevsky’s signature scheme [75] into a provably secure blind signature [65] in the lattice literature. In CRYPTO’23 [70], the Katsumata, Lai, LeGrow, Qin introduced *CSI-Otter*, the first isogeny-based blind signature scheme. It introduces an ingenious use of the quadratic twist of isogeny group actions for the blinding mechanism. Their first scheme achieves a compact signature size of 8 KB. By accepting a larger public key, the size can be reduced to 4 KB.

Beyond the limited availability of suitable blinding techniques, achieving security in the concurrent setting poses an additional challenge. For instance, the construction [65] guarantees security for only up to 7 concurrent signing sessions, while the CSI-Otter scheme [70] supports security for at most logarithmically many concurrent sessions in the security parameter. Although the authors of [70] conjecture that known concurrent attacks in the discrete logarithm setting [17] may not carry over to the isogeny setting due to the limited algebraic structure of group actions, recent concurrent works [52,71] demonstrate that CSI-Otter re-

mains vulnerable in the polynomially-concurrent regime. For practical parameter choices, it becomes forgeable after only *eight* concurrent sessions.

Furthermore, [52] shows that the common *parallel repetitions* of the sigma protocol with (polynomially-)small challenge spaces cannot yield blind signatures secure against polynomially-many concurrent executions. This places a strong limitation on possible blind signature constructions based on alternative post-quantum assumptions such as the linear code equivalence problem or the lattice isomorphism problem [80,54] using in LESS and HAWK’s predecessor respectively. Both the proposals [25,39] also use the parallel repetition in the underlying sigma protocol. In addition, [71] extends the attack methodology to lattice-based blind signatures [6,7], showing that they can be broken with as few as *four* concurrent sessions—without relying on parallel repetition—further calling into question the viability of this approach for broader use.

As a result, we are left with lattice-based blind signatures as the sole viable post-quantum option—an unsatisfactory state, especially in light of the need for diversity in cryptographic assumptions. This highlights the importance of constructing blind signature schemes from alternative post-quantum foundations. This brings us to the main question of this work:

*Can we develop new blinding techniques for the sigma protocol approach
that apply across post-quantum assumptions
and even provably secure under polynomial concurrency?*

1.1 Our Contributions

We contribute to the post-quantum blind signature literature by proposing four frameworks based on cryptographic group actions, each with different security guarantees and trade-offs in efficiency. We collectively call these frameworks **Tanuki(s)** (shuffling **T**hree-move **I**ndistinguishable signing with **U**niform-weight challenge from **K**rypto group **A**ction).

We note that we will often call those schemes or constructions, but they are frameworks that can be instantiated using different group actions. The contributions are as follows:

- **Framework 1:** We introduce a novel blinding technique for a simple Σ -protocol for the graph isomorphism problem. This leads to our first construction, which achieves *sequential security* under the one-more vectorization assumption and *logarithmic concurrent security*. While it has the largest signature size among our schemes, it can be proven secure under the standard group action inverse assumption via a modified OR-proof technique as **Framework 4 (Appendix D)**. For 128-bit security, when instantiated with CSI-FiSh and LESS, the resulting (public key, signature) sizes are (0.06 KB, 8.7 KB) and (13.6 KB, 128 KB), respectively.
- **Framework 2:** We extend Framework 1 by introducing a feature that enables a trade-off between public key and signature size—without weakening the underlying assumption and in a manner agnostic to the group action.

This feature is rare in advanced primitives based on group actions. The resulting scheme maintains sequential security and log-concurrent security under the same assumptions. For 128-bit security, the corresponding (public key, signature) sizes are: CSI-FiSh (0.96 KB, 2.6 KB) and LESS (204 KB, 37.8 KB).

- **Framework 3:** Building on Framework 2, we present the first sigma-protocol-based blind signature scheme with *provable polynomial concurrent security* in the post-quantum setting. This is achieved by introducing an additional randomization layer where the signer and the user generates an ephemeral public key by shuffling and randomizing the original one. The added overhead in signature is only the size of the randomness generating the ephemeral public key (i.e. the size of the secret key). For 128-bit security, instantiations with CSI-FiSh and LESS yield (4.4 KB, 4.5 KB) and (936 KB, 64.7 KB), respectively.

We also provide proof-of-concept implementations in C using CSI-FiSh and LESS, showing that the computational cost is primarily dominated by the cost of the group action.

1.2 Technical Overview and Concurrent Work

A group action is defined over a group $(G, \cdot)$ and a set X via an operation $\star : G \times X \rightarrow X$ that satisfies compatibility: $(g \cdot h) \star x = g \star (h \star x)$. Crucially, our schemes impose *no additional structural requirements* including *commutativity*, and can be instantiated with a broader range of group actions beyond CSIDH-512 (CSI-FiSh). We concretely consider the one underlying the NIST round-two candidate LESS and discuss the application to the group actions underlying the MEDS [39], ALTEQ [25] and HAWK [53] schemes in Remark 3. This contrasts with CSI-Otter, which fundamentally depends on the commutativity of the group action due to its design.

Basic Scheme with log-concurrent security. Equipped with cryptographic group actions, we construct our first scheme in Section 3. We start with a generic Σ -protocol for graph isomorphism [62]. To recap, this protocol proceeds as follows. The prover tries to convince a verifier that it knows $\text{sk} = g \in G$ such that $\text{pk} = g \star x$ without revealing g . In the first step, the prover picks a random $h \leftarrow \$ G$ and sends the commitment $y = h \star x$ to the verifier, who replies with a bit challenge c . The prover replies with $s = h$ in case $c = 0$ and with $s = g \cdot h^{-1}$ otherwise. The verifier checks that $s \star x = y$ if $c = 0$ and $s \star y = \text{pk}$ if $c = 1$. The protocol can be turned into a signature scheme using parallel repetition and the Fiat-Shamir transformation, where the challenge c is computed as the output of a hash function modeled as a random oracle.

Our new blinding technique is that the user can pick random group elements $r_1, \dots, r_\mu \leftarrow \$ G$ and randomize the commitments $y_1, \dots, y_\mu$ by computing $z_i = r_i \star y_i$ for $i \in [\mu]$. Moreover, before computing the challenge c , the user picks a random permutation π over $[\mu]$. The challenge c is then computed as $H(z_{\pi(1)}, \dots, z_{\pi(\mu)} \| m)$. Applying those steps almost ensures the blindness of the scheme. Clearly, the signer might still break blindness using the challenge

$c' = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\mu)})$ sent by the user, exploiting the fact that the permutation does not change the Hamming weight of the output of the hash function. Therefore, we let the hash function used $H : \{0, 1\}^* \rightarrow \{0, 1\}^\mu$ always outputs values with Hamming weight $\mu/2$.

We show that the scheme implementing the above idea is perfectly blind and unforgeable. In more detail, we first show the scheme’s concurrent security for logarithmically many sessions, which is the best we can hope for and in line with the recent attacks [71, 52]. While this result is interesting, our scheme would not be practical as it would only support a couple of signatures before the signer would have to change its signing key.

Can we infer sequential from log-concurrent security? Thus, we explore an interesting observation made in [52] regarding concurrently secure schemes for logarithmically many sessions. The idea is to use such schemes in the sequential setting instead, if a concurrent proof only supports logarithmically many signatures. Specifically, one might speculate that if a scheme achieves ℓ one-more unforgeability for $\ell \geq 2$ in the concurrent setting, it should imply sequential security for any polynomial ℓ . We demonstrate that this is not the case. To prove this, we construct a scheme inspired and based on Fischlin’s approach [59]: 1) The signer’s public key comprises a public key for the signature scheme. 2) The user begins the interaction by committing to their message. 3) The signer signs the commitment using the secret key. 4) The final signature includes a zero-knowledge proof where the user demonstrates knowledge of a valid signature under the signer’s public keys and a commitment that opens to the message. The statement for the proof is extended with a trapdoor witness that can be used when one knows ℓ signatures under distinct messages.

This design ensures blindness via the zero-knowledge proof and ℓ one-more unforgeability through the signature scheme’s security and soundness of the proof system. Although the scheme supports ℓ concurrent sessions, it fails to achieve $\ell + 1$ sequential unforgeability. The adversary can extract one additional signature for one of the public keys and use the trapdoor witness, breaking the unforgeability of the blind signature scheme (cf. Appendix C for more details).

Basic Scheme with sequential security. Consequently, we must provide a separate proof of sequential security for our scheme. Our scheme is sequentially secure in the algebraic group action model for polynomially many signatures. We show that this proof holds under an assumption that resembles the one-more discrete logarithm assumption [15], but for group actions. We call the assumption one-more-vectorization problem. The basic scheme, while being the least efficient of all of our schemes in terms of signature size, can be easily extended using OR-proof techniques to provide sequential security under the group action inverse problem without the need for the interactive one-more assumption. This framework is presented in Appendix D. The idea is to have two public keys, one of which will be used to simulate the protocol while the other one will be the problem instance. The downside of this technique is that the public key and signature size are doubled.

Multi-Key Scheme: finding size-tradeoffs. We introduce a second framework representing a multi-key scheme in Section 4 to address the limitation of the basic scheme’s large signature size. Building on the first scheme, our second scheme is parameterized, allowing for a smaller signature size by introducing additional public keys for the signer – hence the name multi-key. We remark that while this capability, firstly introduced in [50], is well-known for conventional signatures [9,25,39], it is distinctive in the realm of advanced primitives (e.g.[20,19]) including blind signatures. A similar trade-offs in CSI-Otter require relaxations of the underlying assumptions and a specific structure in the underlying group. In contrast, our modification does not compromise the underlying assumption and is agnostic to the group structure.

The starting point for the scheme is a slightly different sigma protocol. Assume that the prover’s public key is $\mathbf{pk} = (g_1 \star x, \dots, g_\kappa \star x)$ and the statement corresponds to proving knowledge of the secret key $\mathbf{sk} = (g_1, \dots, g_\kappa)$ corresponding to $\mathbf{pk}$. The protocol starts with the prover sending a commitment $y = h \star x$ for a random $h \leftarrow \$ G$. The verifier computes a challenge $c \leftarrow \$ [\kappa]$ and sends it to the prover, who replies with $s = h \cdot g_c$. The verifier accepts the proof if $s \cdot \mathbf{pk}_c = y$. As mentioned, by setting $\mathbf{pk} = (x, g \star x)$, the protocol is the same as the proof for graph isomorphism.

Interestingly, our blinding technique applies still. The signer sends μ commitments $Y = (y_1, \dots, y_\mu)$ with $y_i = h_i \cdot x$ to the user. The latter picks values $r_1, \dots, r_\mu \leftarrow \$ G$ and randomizes the commitments by computing $z_i = r_i \star y_i$. The user again picks a permutation $\pi : [\mu] \rightarrow [\mu]$, computes the hash of the permuted commitments $(c_1, \dots, c_\mu) = \mathbf{H}(z_{\pi(1)}, \dots, z_{\pi(\mu)}, m)$, and sends the challenge $C' = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\mu)})$ to the signer. The signer replies with pre-signature $S' = (s'_1, \dots, s'_\mu)$, where the values $s'_i = h_i \cdot g_{c'_i}^{-1}$ correspond to the challenge picked by the user. Once obtained, the user applies the randomization of the pre-signature and computes the values $s_i = r_i \cdot s'_i$ for each $i \in \mu$. The final signature then takes into account the correct permutation of all the values and is of the form $\mathbf{sig} = ((z_{\pi(1)}, \dots, z_{\pi(\mu)}), (s_{\pi(1)}, \dots, s_{\pi(\mu)}))$.

An essential factor omitted in the above description is that we need a similar property for the hash function $\mathbf{H}$ that we introduced for the basic scheme. There, the outputs of the function are values with the same number of ones and zeros. For the multi-key scheme, we need something stronger. Firstly, we need to assume that the number of repetitions μ is divided by the number of public keys and that $\mu = \alpha \cdot \kappa$ for some natural number α . Moreover, we assume that for any pair $i, j \in [\kappa]$, the number of occurrences of i is the same as the number of occurrences of j . With those requirements, we achieve blindness.

Let’s assume the number of public keys κ is larger than the number of repetitions μ . In such a setting, blindness cannot hold as there will be some public keys that will not be used in the generation of a signature. Thus, a malicious signer can link the signing process to a final signature by investigating the public keys used in the signature. The same happens in case the output of the hash function picks one public key more frequently for a given signature, allowing a malicious signer also to break blindness. Note that if we assume that $\mu = \alpha \cdot \kappa$

and restrict the hash using the above condition, we end up in a scenario where each public key is used to answer exactly α challenges for a single signature.

Instantiating the hash function. One open question is how to instantiate such a hash function efficiently. In the case of the basic scheme, the simplest idea would be to add a counter to the message and, while increasing the counter, check if the hash function output has the same number of zeros and ones and use that as the output. Unfortunately, a similar process would not work for this scheme, since we expect $\mu = \alpha \cdot \kappa$ values from the set $[\kappa]$. Instead, we first assign each public key to exactly α values from the set $[\mu]$. Picking a random permutation over $[\mu]$ will solve our problem. A simple construction for this problem is the Fischer-Yates shuffling algorithm, which is compatible with random oracles. Given an input $N = \mu$ using uniform random sampling of values, the algorithm produces a uniformly random shuffling of the values from $[N]$. Thus, we get a uniformly random permutation of the public keys using the above assignment. More importantly, the sampling used for the shuffling can be implemented using standard hash functions, consequently allowing us to use this shuffling algorithm as a random oracle. The main advantage of the scheme is that the more we increase the number of public keys κ , the more we can decrease μ and consequently decrease the signature size sig . On the downside, the OR-proof techniques cannot be easily applied to this scheme. Thus, we prove sequential security under the one-more-vectorization assumption and the algebraic group action model. We can only achieve logarithmically many sessions for concurrent security.

Randomized Multi-Key Scheme: achieving poly-concurrent security.

Our final framework in Section 5 addresses the limitation of the multi-key scheme, which only achieves sequential security. By incurring a modest increase in signature size, we achieve security against polynomially many concurrent signing sessions under the one-more vectorization problem in the algebraic group action model. The main idea is to prevent adversaries from exploiting parallel sessions by adaptively correlating challenge strings with the signer’s initial messages—a core technique and the main concept in concurrent attacks [71,52]. We introduce our countermeasure we refer to as *double blinding*. In this approach, the signer applies randomization and permutation to generate a session-specific (ephemeral) public key pk' , which is embedded in the first message. To bind the user to this specific key, the ephemeral pk' is included in the input to the challenge hash. Upon receiving the challenge, the signer responds relative to pk' , along with the randomness used to derive it. This delays the user’s knowledge of the key-linking structure until after the challenge is committed, thereby preventing adaptive attacks intuitively.

However, naively including pk' in the hash breaks blindness. To fix this, the user further randomizes and permutes pk' into pk'' , and computes the challenge as $c = H(z_{\pi(1)}, \dots, z_{\pi(\mu)}, \text{pk}'', m)$. It is important to emphasize that this “double blinding” technique builds critically on the multi-key scheme (rather than the basic one). It is effective only when the public key contains a sufficiently large number of elements; otherwise, the adversary may still guess the signer’s permutation and reassemble a valid forgery [71,52]. The final signature $\text{sig} =$

$((z_{\pi(1)}, \dots, z_{\pi(\mu)}), (s_{\pi(1)}, \dots, s_{\pi(\mu)}), (t_{\sigma(1)} \cdot t'_{\sigma'(\sigma(1))}, \dots, t_{\sigma(\kappa)} \cdot t'_{\sigma'(\sigma(\kappa))}), \sigma' \circ \sigma)$ includes the randomized commitments z , the solutions s , the randomizations of the public keys t and the permutations/shuffling $\sigma' \circ \sigma$ of the public key. Compared to the multi-key version, the signature size is increased by κ elements from G and a representation for a permutation on $[\kappa]$.

Related Works. In Appendix A we provide a discussion of related work on post-quantum blind signatures and the use of group actions.

Concurrent Works. Two recent works have appeared on the IACR ePrint Archive [73,56], both of which can be seen as follow-ups to CSI-Otter, adopting similar blinding techniques. The work of [73] targets the MEDS signature scheme [39], which also admits a group action structure, and replaces the quadratic twist with a *negative transposition*. The second work [56] extends CSI-Otter to a more general group action framework. In comparison, their schemes are less efficient than our basic one Framework 1 in terms of the number of group elements included in the signature: 4λ for [73], 8λ for [56] and $\mu \approx 2.1\lambda$ in case of our Framework 1. Furthermore, their schemes do not support the key-signature size trade-off achieved by our second construction (Framework 2), nor do they offer provable security in the polynomially-concurrent setting, a feature that is provided by our third construction (Framework 3).

2 Preliminaries

Notations. We let $\mathbb{N}$ denote the set of natural numbers. For $n \in [n]$, we denote the set $\{1, \dots, n\}$ as $[n]$. For $\kappa \in \mathbb{N}$ we denote with S_κ the set of all the permutations of $[\kappa]$. If $\sigma, \sigma' \in S_\kappa$ are permutations then we denote with $\sigma \circ \sigma'$ their composition (as functions). For two vectors $\mathbf{b}_1, \mathbf{b}_2$, we denote $\mathbf{b}_1 =_{\mathbf{B}} \mathbf{b}_2$ if the zeros in the vectors are at the same positions. For an element $b \in \{0, 1\}$, we define its conjugation as $\bar{b} = 1 - b \in \{0, 1\}$. For a group action $(G, X, \star)$ and $x \in X$, we extend the action to an array $(g_1, \dots, g_n) \star x := (g_1 \star x, \dots, g_n \star x) \in X^n$. For a list $C = [c_1, \dots, c_n]$ with $c_i \in G$ and set element $x \in X$ we also define $C \star x := [c_1 \star x, \dots, c_n \star x]$.

2.1 Cryptographic Group Actions

The study of post-quantum group actions has seen growing interest in recent years. This line of research began in the commutative setting, notably through isogeny-based constructions [35,4], and has since demonstrated broad applicability to advanced cryptographic primitives [4,20,51,19,74,78]. Parallel or the inspired developments [9,39,25,11,33,14,12,34] have emerged in the non-commutative setting, with increasing attention in both code-based and lattice-based cryptography. Along with the rapid development, a number of abstract models have been introduced to streamline security analyses and to formalize hardness assumptions or impossibility results [4,30,55,12].

We describe the cryptographic group action used in this work. Unlike the setting considered in [4], our setting does not require the group to be commutative.

Consequently, there is no known general subexponential algorithm for solving the underlying assumptions. We begin by recalling the basic definitions.

Definition 1 (Group action). Let $(G, \cdot)$ be a group. We say G acts on a set X by a function $\star : G \times X \rightarrow X$ or $(G, X, \star)$ is a group action if it satisfies the following properties. 1) Identity: for any $x \in X$, we have $e \star x = x$ where e is the identity element of G . 2) Compatibility: for any $g, h \in G$ and any $x \in X$, we have $(gh) \star x = g \star (h \star x)$.

Definition 2 (Cryptographic Group Action). Let $(x, G, X, \star)$ be given where $(G, X, \star)$ defines a group action and $x \in X$. We say that $(x, G, X, \star)$ is a cryptographic group action if there exist computationally efficient algorithms for the following tasks:

- **Unique set element representation:** Given any element $x \in X$, compute a unique representation of x .
- **Unique group element representation:** Given any element $g \in G$, compute a unique representation of g .
- **Group sampling:** Sampling over G uniformly at random (or statistically).
- **Group element inversion:** Given any $g \in G$, compute its inverse g^{-1} .
- **Group action:** Given any $x \in X$ and $g \in G$, compute $g \star x$.

2.2 Hardness Assumptions and the Algebraic Group Action Model

We introduce the assumptions we considered in this work.

Definition 3 (Group action inverse problem (GAIP/GA-DLog)). Let $\Pi = (x, G, X, \star)$ be a cryptographic group action. The group action inverse problem asks, given $y = g \star x$ where $g \leftarrow \$ G$ to find an element $g' \in G$ such that $g' \star x = y$. We define the advantage of a p.p.t. adversary $\mathcal{A}$ in solving GAIP with respect to Π as $\text{Adv}_{\Pi, \mathcal{A}}^{\text{GAIP}}(\lambda) := \Pr[\mathcal{A}(y) \in G \text{ such that } \mathcal{A}(y) \star x = y]$, where the probability is over the randomness of sampling over G and the internal randomness of $\mathcal{A}$.

Definition 4 (One-More-Vectorization Problem (OMVP)). Let $\Pi = (x, G, X, \star)$ be a cryptographic group action. We define the one-more vectorization problem game OMVP with respect to a p.p.t. adversary $\mathcal{A}$ in figure 1. We define the advantage of $\mathcal{A}$ in OMVP with respect to Π as $\text{Adv}_{\Pi, \mathcal{A}}^{\text{OMVP}}(\lambda) := \Pr[\text{OMVP}^{\mathcal{A}}(\lambda) = 1]$, where the probability is over the randomness of sampling over G in the game and the internal randomness of $\mathcal{A}$.

Definition 5 (Algebraic Algorithm). Let the cryptographic group action $\Pi = (x, G, X, \star)$ be given. We say that an algorithm $\mathcal{A}$ is algebraic if, for any set element $y \in X$ that it outputs, it also outputs a group element g and an index $i \in \mathbb{N}$, i.e., $(y, (g, i)) \leftarrow \$ \mathcal{A}(\vec{x})$ such that $y = g_i \star x_i$.

$\text{OMVP}_H^A(\lambda)$	$\text{dLog}(y, z)$	$\text{Chall}()$
$C = []; k = 0;$	$k = k + 1;$	$g \leftarrow \mathfrak{s} G; y \leftarrow g \star x;$
$(x, X, G, \star) \leftarrow \Pi$	if $\exists g$ s.t. $g \star y = z$ then	$C = C \parallel [g]$
$C' \leftarrow \mathcal{A}^{\text{Chall}, \text{dLog}}(x, X, G, \star)$	return g	return y
return $(C \star x = C' \star x \wedge k < C)$	else return $\perp$	

Fig. 1. The game OMVP and the oracles Chall, dLog.

For an adversary $\mathcal{A}$ that has access to oracles during its runtime, we impose the above restriction on all group elements that it outputs to an oracle. Similarly, all group elements that $\mathcal{A}$ receives through oracle interactions are treated as inputs to $\mathcal{A}$; hence, such group elements become part of $\vec{x}$ when $\mathcal{A}$ outputs set elements (and hence algebraic coefficients) at a later point. In the *algebraic group action model* (AGAM), all algorithms are treated as algebraic algorithms. Moreover, we define the *running time* of an algorithm $\mathcal{A}$ in the AGAM as the *number of group action operations* that $\mathcal{A}$ performs. The model is an analogue to algebraic group model [61] and it has been considered in [55] in the commutative setting. In lattice literature, similar concept can be found in [57] algebraic one more MLWE.

2.3 Blind Signature Schemes

We will now recall the notion of blind signatures. We focus on a subclass called three-move blind signatures, where the exchange between the user and signer is finished in three moves, similar to Sigma protocols.

Definition 6. A three-moves blind signature scheme $\text{BS} = (\text{KeyGen}, \text{Signer}, \text{User}, \text{Verify})$ consists of the following p.p.t. algorithms:

Key generation. On input the security parameter 1^λ , the probabilistic algorithm KeyGen outputs a public key pk and a secret key sk .

Signer. The $\text{Signer} = (\text{Signer}_1, \text{Signer}_2)$ consists of two algorithms:

$\text{Signer}_1(\text{sk})$: on input a secret key sk , the probabilistic algorithm Signer_1 outputs an internal state $\text{st}_{\text{Signer}}$ and a commitment R .

$\text{Signer}_2(\text{sk}, c, \text{st}_{\text{Signer}})$: on input a challenge $c \in \mathcal{C}$, a secret key sk and an internal state st , the algorithm Signer_2 outputs a response s .

User. The $\text{User} = (\text{User}_1, \text{User}_2)$ consists of two algorithms:

$\text{User}_1(\text{pk}, m, R)$: on input a public key pk , a message m and a signer commitment R , the probabilistic algorithm User_1 outputs a challenge $c \in \mathcal{C}$ and an internal state st_{User} .

$\text{User}_2(\text{pk}, s, \text{st}_{\text{User}})$: on input a public key pk , a signer response s and an internal state st_{User} , the probabilistic algorithm User_2 outputs a signature sig on message m .

Verification. On input a public key pk , a message m , and a signature sig , the verification algorithm Verify outputs 1 to indicate the prover is valid, and 0 otherwise.

We say that a three-moves blind signature scheme BS is correct if for every λ , for every key pair $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\lambda)$, for every message and signature pair (m, sig) generated from the interaction in figure 4 between an user and a signer, we have $\text{Verify}(\text{pk}, m, \text{sig}) = 1$. We will omit “three-moves” and will say blind signature for brevity.

We consider blind signatures secure if they have the following two security properties. On the one hand, we want the signature output by the user to be not linkable to its identity: this is called the *blindness* property. On the other hand, after ℓ interactions with the signer, we require that the user cannot output more than ℓ valid signatures. This property is called *one-more-unforgeability*. We formalize it using the definition from [68].

Definition 7 (Concurrent ℓ -one-more unforgeability). A blind signature BS is ℓ -one-more unforgeable in the concurrent setting if there exists a negligible function $\text{negl}(\lambda)$ such that for all λ and all PPT adversaries $\mathcal{A}$, the following holds: $\text{Adv}_{\mathcal{A}, \ell}^{\text{cOMUF}}(\lambda) \leq \text{negl}(\lambda)$, where $\text{Adv}_{\mathcal{A}, \ell}^{\text{cOMUF}}(\lambda) = \Pr[\text{cOMUF}(\lambda) = 1]$ and cOMUF is the experiment defined in Figure 2.

Definition 8 (Sequential ℓ -one-more unforgeability). A blind signature BS is ℓ -one-more unforgeable in the sequential setting if there exists a negligible function $\text{negl}(\lambda)$ such that for all λ and all PPT adversaries $\mathcal{A}$, the following holds: $\text{Adv}_{\mathcal{A}, \ell}^{\text{sOMUF}}(\lambda) \leq \text{negl}(\lambda)$, where $\text{Adv}_{\mathcal{A}, \ell}^{\text{sOMUF}}(\lambda) = \Pr[\text{sOMUF}(\lambda) = 1]$, where sOMUF is similar to the experiment cOMUF defined in Figure 2 except with an additional restriction. The adversary is not allowed to query the Sign_1 oracle if there already exists an open session, i.e., the oracle returns $\perp$ if there exists a $k \leq \text{sid}$ for which $\text{sess}_k = \text{open}$.

Definition 9 (Malicious-Key Blindness). A blind signature scheme BS is blind under malicious key if there exists a negligible function $\text{negl}(\lambda)$ such that for all λ and all PPT adversaries $\mathcal{A}$, the following holds: $\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) \leq \text{negl}(\lambda)$, where $\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) = |\Pr[\text{ExpBlind}(\lambda) = 1] - \frac{1}{2}|$ and the experiment ExpBlind is defined in Figure 3. When $\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) = 0$, we say the blind signature scheme BS has perfect blindness.

3 First Framework

In this section, we introduce our novel blind signature framework based on generic group actions. We prove that the scheme achieves perfect blindness, log-concurrent security, and sequential security. We describe our construction in Framework 1. The high-level idea is to build upon a generic sigma protocol for

cOMUF(λ)	Sign ₁ ($\cdot$)
$(sk, pk) \leftarrow \text{KeyGen}(\lambda)$	$sid \leftarrow sid + 1$
$sid \leftarrow 0$	$sess_{sid} \leftarrow \text{open}$
$((m_1, sig_1), \dots, (m_\ell, sig_\ell)) \leftarrow \mathcal{A}^{\text{Sign}_1, \text{Sign}_2}(pk)$	$(st_{sid}, R_{sid}) \leftarrow \text{Signer}_1(sk)$
if $\exists i \neq j : m_i = m_j$: return 0	return (sid, R_{sid})
if $\exists i \in [\ell] : \text{Verify}(pk, m, sig_i) = 0$: return 0	
$Q_{\text{Sign}_2} \leftarrow \{k sess_k = \text{closed}\} $	Sign ₂ (sid, c)
if $\ell \geq Q_{\text{Sign}_2} + 1$: return 1	if $sess_{sid} \neq \text{open}$: return $\perp$
return 0	$sess_{sid} = \text{closed}$
	$s_{sid} \leftarrow \text{Signer}_2(sk, st_{sid}, R_{sid}, c)$
	return s_{sid}

Fig. 2. Unforgeability experiment cOMUF.

ExpBlind(λ)	Init($\tilde{m}_0, \tilde{m}_1, \tilde{pk}$)
$b \leftarrow \{0, 1\}$	$m_0 \leftarrow \tilde{m}_0, m_1 \leftarrow \tilde{m}_1$
$b_1 \leftarrow b; b_2 \leftarrow 1 - b$	$pk \leftarrow \tilde{pk}$
$b' \leftarrow \mathcal{A}^{\text{Init}, U_1, U_2}(\lambda)$	$sess_1 \leftarrow sess_2 \leftarrow \text{init}$
return $b = b'$	
	U ₂ (sid, s)
U ₁ (sid, R)	if $sess_{sid} \neq \text{open}$: return $\perp$
if $sid \notin \{1, 2\} \vee sess_{sid} \neq \text{init}$: return $\perp$	$s_{sid} \leftarrow s$
$sess_{sid} \leftarrow \text{open}$	$sig_{b_{sid}} \leftarrow \text{User}_2(pk, R_{sid}, c_{sid}, s_{sid}, st_{sid})$
$R_{sid} \leftarrow R$	if $sess_1 = sess_2 = \text{closed}$:
$(c_{sid}, st_{sid}) \leftarrow \text{User}_1(pk, R_{sid}, m_{b_{sid}})$	if $sig_0 = \perp \vee sig_1 = \perp$: return $(\perp, \perp)$
return (sid, c_{sid})	return (sig_0, sig_1)
	return (sid, closed)

Fig. 3. Malicious-Key Blindness experiment ExpBlind.

the graph-isomorphism-type problem. After the signer sends a commitment, the user “masks” it by applying random group elements. Subsequently, the user also masks the challenge string by applying a random permutation. However, since a permutation preserves the Hamming weight of a string, an adversary could potentially exploit this invariant to compromise the blindness of the scheme for distinct sessions. To hide the information, we restrict the challenge space to strings of which the Hamming weight is exactly half of the length.

Theorem 1. *The blind signature presented in Framework 1 has correctness.*

Let $(x, X, G, \star)$ be a cryptographic group action. Let $\mu = \mu(\lambda)$ be functions in the security parameter λ to even numbers. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^\mu$ be a hash function where the output has Hamming weight $\mu/2$.

KeyGen(λ): sample group element $g \leftarrow G$, output $\text{sk} = g$ and $\text{pk} = g \star x$

Signer₁(sk): sample group elements $h_1, \dots, h_\mu \leftarrow G$. Compute $y_i = h_i \star x$ for each $i \in [\mu]$. Return $y = (y_1, \dots, y_\mu)$ to the user.

User₁(pk, m, y): parse y as $(y_1, \dots, y_\mu)$, randomly sample group elements $r_1, \dots, r_\mu \leftarrow G$ and a permutation π over $[\mu]$. Compute $z_i = r_i \star y_i$. Compute the challenge $c = H(z_{\pi(1)}, \dots, z_{\pi(\mu)} \| m)$, denoted as $(c_1, \dots, c_\mu)$. Set $c' = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\mu)})$. Send c' to Signer.

Signer₂(sk, c'): Parse $c' = (c'_1, \dots, c'_\mu)$. For each $i \in [\mu]$, set $s'_i = \begin{cases} h_i & \text{if } c'_i = 0 \\ g \cdot h_i^{-1} & \text{if } c'_i = 1 \end{cases}$. Send $s' = (s'_1, \dots, s'_\mu)$ to the User.

User₂(pk, s'): For each $i \in [\mu]$, return $\perp$ if $y_i \neq s'_i \star x$ when $c'_i = 0$, or if $y_i \neq s'^{-1}_{i-1} \star \text{pk}$ when $c'_i = 1$; otherwise, set $s_i = \begin{cases} r_i \cdot s'_i & \text{if } c'_i = 0 \\ s'_i \cdot r_i^{-1} & \text{if } c'_i = 1 \end{cases}$ for each $i \in [\mu]$. Output signature $\text{sig} = ((z_{\pi(1)}, \dots, z_{\pi(\mu)}), (s_{\pi(1)}, \dots, s_{\pi(\mu)}))$. Abort if it is invalid.

Verify(pk, m, sig): Parse $\text{sig} = ((\tilde{z}_1, \dots, \tilde{z}_\mu), (\tilde{s}_1, \dots, \tilde{s}_\mu))$ and compute $(\tilde{c}_1, \dots, \tilde{c}_\mu) = H(\tilde{z}_1, \dots, \tilde{z}_\mu \| m)$. Output 1 if for every $i \in [\mu]$ it holds that $\tilde{s}_i \star x = \tilde{z}_i$ when $\tilde{c}_i = 0$, and $\tilde{s}_i \star \tilde{z}_i = \text{pk}$ when $\tilde{c}_i = 1$; otherwise, output 0.

Framework 1: Group Action Blind Signature

Proof. We use the same notations as described in Framework 1. The abort in the User₂ step will not occur if both parties follow the scheme specification. Given that $s'_i = \begin{cases} h_i & \text{if } c'_i = 0 \\ g \cdot h_i^{-1} & \text{if } c'_i = 1 \end{cases}$, it follows that $y_i = \begin{cases} s'_i \star x & \text{if } c'_i = 0 \\ s'^{-1}_{i-1} \star \text{pk} & \text{if } c'_i = 1 \end{cases}$. It suffices to show that the verification always outputs 1. Since $c'_i = c_{\pi^{-1}(i)}$ for each $i \in [\mu]$, it implies that $c'_{\pi(i)} = c_i$. Therefore,

$$\begin{cases} \text{if } c_i = 0, & \tilde{s}_i \star x = r_{\pi(i)} \cdot s'_{\pi(i)} \star x = r_{\pi(i)} \star y_{\pi(i)} = z_{\pi(i)}, \\ \text{if } c_i = 1, & \tilde{s}_i^{-1} \star \text{pk} = (r_{\pi(i)} \cdot s'^{-1}_{\pi(i)}) \star \text{pk} = r_{\pi(i)} \star y_{\pi(i)} = z_{\pi(i)}. \end{cases}$$

This proves the correctness of the scheme.

Remark 1 (Recoverable commitments.). We remark that to simplify the proof in the algebraic group action model, we include the set elements in the signature. In fact, in Framework 1, we can modify the User₂ algorithm such that the user returns $((c_1, \dots, c_\mu), (s_{\pi(1)}, \dots, s_{\pi(\mu)}))$ as the signature. This modification allows any public verifier to compute $z_{\pi(i)}$ using $s_{\pi(i)}$ and (x, pk) . In this case,

the verification process merely involves verifying the hash output. This approach results in a more compact signature. We utilize this alternative to estimate the performance in Section 6.

3.1 Security Proofs

We begin with showing the blindness property. We emphasize that we do not assume the group action to be free, meaning that $g_1 \star x = g_2 \star x$ does not necessarily imply $g_1 = g_2$ (e.g., the group action defined by [10] is not free). As a result, it is insufficient to prove blindness merely by showing the existence of randomness that, for a given signer’s view, produces a specific signature. Instead, we construct a bijection between the randomness used in two signing sessions. Since Framework 1 is a special case of Framework 2, we refer to Theorem 6 for the complete proof and only state the theorem below.

Theorem 2. *The blind signature presented in Framework 1 has perfect blindness.*

Log-Concurrent Security.

We now show the log-concurrent security in the algebraic group action model and the random oracle model. To make the proof more accessible, we introduce the *binary-parallel ROS* problem (bpROS), which, roughly speaking, captures the residual cases where a reduction to the one-more vectorization problem (OMVP) fails. That is, we reduce OMUF to a combination of OMVP and bpROS. Then, we show an information-theoretic bound on the adversary’s success probability in solving bpROS in the random oracle model, which in turn yields the desired log-concurrent security bound.

We emphasize that the introduction of the bpROS problem is primarily for pedagogical clarity. In subsequent sequential security proofs and in later sections, we prove the corresponding information-theoretic bounds *directly*, without explicitly introducing their own ROS-type problems. This avoids repetition and allows us to streamline the presentation, especially since different schemes and security notions will require their own ROS variants. We also remark that our bpROS problem here is orthogonal to those defined in the classical setting (e.g. [17]). We define the bpROS problem below, generalizing the formulation from [71].

Definition 10 (Binary-Parallel-ROS Problem.). *Given the parameters $\mu \in 2\mathbb{N}, \ell \in \mathbb{N}$, let $H_{\text{BP-ROS}} : G^{\mu \times (2+\ell\mu)} \times \{0,1\}^* \rightarrow \{0,1\}^\mu$ be a random oracle where the output is of Hamming weight $\mu/2$. The binary-parallel-ROS problem is given access to $H_{\text{BP-ROS}}$, and the adversary outputs $\ell + 1$ matrices $\mathbf{A}_1, \dots, \mathbf{A}_{\ell+1} \in G^{\mu \times (2+\ell\mu)}$, strings $\text{aux}_1, \dots, \text{aux}_{\ell+1} \in \{0,1\}^*$, and a binary string and $\mathbf{c} = (0 \ 1 \ \mathbf{c}')^\top$ where $\mathbf{c}' \in \{0,1\}^{\ell\mu}$. We say $\mathcal{A}$ wins if $\mathcal{A}$ ’s output satisfies the following conditions:*

1. *Each row of $\mathbf{A}_1, \dots, \mathbf{A}_{\ell+1}$ has one and only one non-zero entry; and*

2.

$$\begin{bmatrix} \mathbf{A}_1 \\ \vdots \\ \mathbf{A}_\ell \\ \mathbf{A}_{\ell+1} \end{bmatrix} \cdot \mathbf{c} =_{\mathbf{B}} \begin{bmatrix} \mathbf{H}_{\text{BP-ROS}}(\mathbf{A}_1, \text{aux}_1) \\ \vdots \\ \mathbf{H}_{\text{BP-ROS}}(\mathbf{A}_\ell, \text{aux}_\ell) \\ \mathbf{H}_{\text{BP-ROS}}(\mathbf{A}_{\ell+1}, \text{aux}_{\ell+1}) \end{bmatrix}, \quad (1)$$

where, within the inner product, for every $g \in G$, we define $g \cdot 0 := 0$.

We denote the advantage of the adversary by $\text{Adv}_{\ell, \lambda'}^{\text{BP-ROS}}(\mathcal{A}) := \Pr[\mathcal{A} \text{ wins.}]$

Looking ahead, the parameter μ corresponds to the number of parallel repetitions in our scheme and ℓ corresponds to the number of concurrent sessions. Every matrix $\mathbf{A}_i$ represents an array of set elements, which obtained through the acting the corresponding group element on one of the given set elements. We first show the generic hardness of the binary-parallel-ROS problem.

Theorem 3 (Upper Bound). *For any computationally-unbounded binary-parallel-ROS problem adversary $\mathcal{A}$ with Q random oracle query, we have*

$$\text{Adv}_{\ell, \mu, \mathcal{A}}^{\text{BP-ROS}}(\lambda) \leq \binom{Q}{\ell+1} \frac{2^{\ell\mu}}{\binom{\mu}{\mu/2}^{\ell+1}}.$$

Proof. If $\mathcal{A}$ makes fewer than $\ell+1$ queries to the oracle, the probability that $\mathcal{A}$ wins is less than $1/\binom{\mu}{\mu/2}$ due to the condition Eq. (1) and $\mathbf{H}_{\text{BP-ROS}}$ being a random oracle. Hence, we may assume $\mathcal{A}$ makes at least $\ell+1$ queries. We take an arbitrary set of $\ell+1$ queries from $\mathcal{A}$ to $\mathbf{H}_{\text{BP-ROS}}$, denoted by $(\mathbf{A}_1, \text{aux}_1), \dots, (\mathbf{A}_{\ell+1}, \text{aux}_{\ell+1})$. By considering Eq. (1) over $\mathbb{F}_2$, the probability that there exists a $\mathbf{c}' \in \{0, 1\}^{\ell\mu}$ such that Eq. (1) holds is less than $2^{\ell\mu} / \binom{\mu}{\mu/2}^{\ell+1}$. This is because the right-hand side of Eq. (1) follows the uniform distribution over the codomain of $\mathbf{H}_{\text{BP-ROS}}$, while $\mathbf{c}'$ gives at most $2^{\ell\mu}$ combinations. Hence, the probability that Eq. (1) is solvable is less than $\frac{2^{\ell\mu}}{\binom{\mu}{\mu/2}^{\ell+1}}$. By the union bound, we prove the result.

Theorem 4. *Given a p.p.t.adversary $\mathcal{A}$ against the blind signature in Framework 1 over the random oracle model and the algebraic group action model, there exists an OMVP adversary $\mathcal{B}_1$ or a binary-parallel-ROS adversary $\mathcal{B}_2$ such that*

$$\text{Adv}_{\mathcal{A}, \ell}^{\text{COMUF}}(\lambda) \leq \text{Adv}_{\ell, \mu, \mathcal{B}_1}^{\text{BP-ROS}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{OMVP}}(\lambda).$$

running in polynomial in λ, ℓ, μ .

Proof. We prove the result by first introducing a game hop.

Game $\mathbf{G}_0$: is the original concurrent OMUF experiment. Say $\mathcal{A}$ makes ℓ signing queries. During the game, $\mathcal{A}$ obtains $2+\ell\mu$ set elements $(x, \text{pk}, x_{1,1}, x_{1,2}, \dots, x_{\ell, \mu})$, where for each $j \in [\ell]$, the set $\{x_{j,i}\}_{i \in [\mu]}$ is obtained from the j -th signing session returned by Signer_1 . When $\mathcal{A}$ outputs the $\ell+1$ forgeries $\{(m_j, (\overrightarrow{\text{com}}_j, \overrightarrow{\text{resp}}_j))\}_{j \in [\ell+1]}$, where for each $j \in [\ell+1]$, we denote $\overrightarrow{\text{com}}_j = (z_{j,1}, \dots, z_{j,\mu})$ and $\overrightarrow{\text{resp}}_j =$

$(s_{j,1}, \dots, s_{j,\mu})$. Additionally, $\mathcal{A}$ provides the pairs $\{(g_{j,i}, \text{ind}_{j,i})\}_{i \in [\mu], j \in [\ell+1]}$, where $g_{j,i} \in G$ represents how the set element $z_{j,i}$ is computed.

Game G_1 : same as G_0 except that we simulate the random oracle via the standard lazy sampling method and add an abort condition at the end of the game. To simulate the random oracle, on input a set vector $\vec{\text{com}}$ and a message m , we record the representation for each set element within $\vec{\text{com}}$ and reply with a string uniformly randomly from the range if the query has not been made before.

After obtaining the forgeries at the end of the experiment, we define the matrices $\mathbf{A}_j \in G^{\mu \times \ell\mu}$ for each $j \in [\ell+1]$ where for each $i \in [\mu]$ where each entry is zero except for having $g_{j,i}$ at the $(j, \text{ind}_{j,i})$ -th position. We abort the game if the equation

$$\mathbf{A}_i \cdot (0, 1, c'_{j,1}, c'_{j,2}, \dots, c'_{j,\mu})^\top =_{\mathbf{B}} \mathbf{H}(\vec{\text{com}}_j \| m_j) \quad (2)$$

holds for each $j \in [\ell+1]$ where $c'_{j,i}$ represents the i -th challenge bit received from User_1 at the j -th signing session. The random oracle simulation does not change the distribution. The modification of the game does not reduce the advantage of the adversary unless the adversary can solve for the parallel-binary ROS problem. The parallel-binary ROS problem adversary $\mathcal{B}_1$ is constructed as follows. $\mathcal{B}_1$ simulates the OMUF experiment by itself, simulates $\mathbf{H}_{\text{BP-ROS}}$ using $\mathbf{H}$ described as in the construction, and interacts with $\mathcal{A}$. Once $\mathcal{A}$ returns the forgeries $\mathcal{B}_1$ establishes the relations as Eq. (2). Hence, we have $\text{Adv}_{\mathcal{A}, \ell}^{\text{COMUF}}(\lambda) \leq \text{Adv}_{\ell, \mu}^{\text{BP-ROS}}(\mathcal{B}_1)$.

Given an adversary $\mathcal{A}$ in G_1 , we construct an OMVP adversary $\mathcal{B}_2$ as follows. First, $\mathcal{B}_2$ replaces the public key generation with a challenge instance from OMVP. Let $i \in [\ell]$. To simulate the i -th signing query, $\mathcal{B}_2$ generates the Signer_1 message $\vec{\text{com}}_i$ by invoking the challenge oracle μ times. After obtaining the User_1 message (i.e., a challenge string), the algorithm $\mathcal{B}_2$ runs for each $j \in [\mu]$ to obtain:

$$s'_{j,i} = \begin{cases} \mathcal{O}_{\text{dlog}}(x, x_{j,i}) & \text{if the challenge bit is 0;} \\ \mathcal{O}_{\text{dlog}}(x_{j,i}, \text{pk}) & \text{if the challenge bit is 1.} \end{cases} \quad (3)$$

Then, $\mathcal{B}_2$ returns $(s'_{j,i})_{i \in [\mu]}$ to simulate the Signer_2 output.

$\mathcal{B}_2$ simulates the random oracle and imposes the abort condition as in G_1 . The simulation is perfect as every element follows the same distribution.

After obtaining the forgeries, claim that $\mathcal{B}_2$ can extract a witness $g \in G$ such that $g \star x = \text{pk}$. To see this, we separate it into 3 cases with respect to the occurrence of the inconsistency in Eq. (2).

Case 1: Inconsistency at the first entry of Eq. (2). In this case, for some $J \in [\ell+1], I \in [\mu]$, we have $g_{J,I} \in G$ such that $g_{J,I} \star x = z_{J,I}$ from the representation of $z_{J,I}$, and $s_{I,J} \in G$ such that $s_{I,J}^{-1} \star \text{pk} = z_{J,I}$ from the forgery. We can obtain the desired element in this case.

Case 2: Inconsistency at the second entry of Eq. (2). The case is similar to the previous one. In this case, for some $J \in [\ell+1], I \in [\mu]$, we have $g_{J,I} \in G$

such that $g_{J,I} \star \text{pk} = z_{J,I}$ from the representation of $z_{J,I}$, and $s_{J,I} \in G$ such that $s_{J,I} \star x = z_{J,I}$ from the forgery. We can obtain the desired element in this case.

Case 3: The rest of the cases. Since Eq. (2) does not hold for some $I \in [\ell+1]$, say the set element by $c'_{J,I} \neq (\text{H}(\overrightarrow{\text{com}}_J \| m_J))_I$ for some $I \in [\mu]$. From the oracle query for the input $z_{J,I}$, we have an element g' such that $g'^{-1} \star z_{J,I}$ or $g' \star z_{J,I}$ is either x or pk . From the forgery, we have $s_{J,I} \star z_{J,I}$ or $s_{J,I}^{-1} \star z_{J,I}$ is either pk or x . We can obtain the desired group element in this case.

Hence, we have $\text{Adv}_{\mathcal{A},\ell}^{\text{cOMUF}}(\lambda) \leq \text{Adv}_{\ell,\mu,\mathcal{B}_1}^{\text{BP-ROS}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{OMVP}}(\lambda)$. $\square$

Remark 2 (Log-concurrent Security). By combining Theorems 3 and 4, we obtain the following upper bound on the adversary's advantage: $\text{Adv}_{\mathcal{A},\ell}^{\text{cOMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}_2}^{\text{OMVP}}(\lambda) + \binom{Q}{\ell+1} \frac{2^{\ell\mu}}{\binom{\mu}{\mu/2}^{\ell+1}}$.

To analyze the asymptotic behavior, by Lemma 3 we have $\binom{\mu}{\mu/2} \approx \frac{2^\mu}{\sqrt{\pi\mu/2}}$. Assuming the number of oracle queries $Q = \text{poly}(\lambda)$ and the number of repetitions μ grows linearly in λ , we see that setting the number of concurrent sessions $\ell = \log(\lambda)$ makes the second term negligible.

Sequential One-more Unforgeability.

In this subsection, we prove sequential one-more unforgeability of Framework 1 in the random oracle model and algebraic group action model for polynomial ℓ . We base our proof on the assumption that the one more vectorization problem (OMVP) is hard. To simplify the proof, we opt not to define the explicit ROS problem tailored for this case. However, the fundamental idea of demonstrating its hardness is implicitly used in the proof.

Theorem 5. *Given a p.p.t. adversary $\mathcal{A}$ against the blind signature construction Framework 1 over the random oracle model and the algebraic group action model using Q random oracle queries, there exists an OMVP adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\mathcal{A},\ell}^{\text{sOMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + \frac{Q + \ell + 1}{\binom{\mu}{\mu/2}} + \frac{2^\mu \binom{Q}{2}}{\binom{\mu}{\mu/2}^2}$$

where $\mathcal{B}$ runs polynomially in λ, μ, ℓ .

Proof. We use a similar strategy of [69]. Let $\mathcal{A}$ be a p.p.t. algebraic adversary against the sequential ℓ one-more unforgeability experiment. We denote the challenge and Dlog oracles in the OMVP game by **Chall** and **dLog** respectively. We construct an OMVP adversary $\mathcal{B}$ with access to $\mathcal{A}$ proceeding as follows.

1. $\mathcal{B}$ invoke $\mathcal{A}$ with the public parameter $(x, X, G, \star)$ obtained from the OMVP game. $\mathcal{B}$ also simulate the random oracle via the standard lazy sampling method. That is, on input a set vector $\overrightarrow{\text{com}}$ and a message m , $\mathcal{B}$ records the representation for each set element within $\overrightarrow{\text{com}}$, reply with a string uniformly randomly from the codomain, and maintain the query history.
2. $\mathcal{B}$ provides the public key to $\mathcal{A}$ by querying the oracle $\text{pk} \leftarrow \text{Chall}()$.

3. For $j \in [\ell]$, $\mathcal{B}$ simulates in response to $\mathcal{A}$'s signing queries as follows:
 - $\mathcal{B}$ generates the Signer_1 message by invoking the challenge oracle μ times for which we denote by $x_{j,i} \leftarrow \text{Chall}()$ for each $j \in [\mu]$.
 - $\mathcal{B}$ generates the Signer_2 message by using the dLog oracle. In particular, $\mathcal{B}$ simulates the response with

$$s'_{j,i} = \begin{cases} \text{dLog}(x, x_{j,i}) & \text{if } c'_{j,i} = 0 \\ \text{dLog}(\text{pk}, x_{j,i})^{-1} & \text{if } c'_{j,i} = 1 \end{cases}$$

where $c'_{j,i}$ represents the i -th challenge bit received from User_1 at the j -th session. (We remark that in this case it suffices to solve the GAIP problem between x and pk for solving OMVP.)

4. $\mathcal{B}$ receives the $\ell + 1$ forgeries $\{(m_j, (\overrightarrow{\text{com}}_j, \overrightarrow{\text{resp}}_j))\}_{j \in [\ell+1]}$, where for each $j \in [\ell + 1]$, we denote $\overrightarrow{\text{com}}_j = (z_{j,1}, \dots, z_{j,\mu})$ and $\overrightarrow{\text{resp}}_j = (s_{j,1}, \dots, s_{j,\mu})$. Additionally, $\mathcal{A}$ provides the pairs $\{(g_{j,i}, \text{ind}_{j,i})\}_{i \in [\mu], j \in [\ell+1]}$, where $g_{j,i} \in G$ represents how the set element $z_{j,i}$ is computed with respect to the given set $(x, \text{pk}, x_{1,1}, x_{1,2}, \dots, x_{\ell,\mu})$.
5. To find $g \in G$ such that $g \star x = \text{pk}$, $\mathcal{B}$ checks when the following equation does not hold

$$\mathbf{A}_j \cdot (0, 1, c'_{j,1}, c'_{j,2}, \dots, c'_{j,\mu})^\top =_{\mathbf{B}} \text{H}(\overrightarrow{\text{com}}_j \| m_j) \quad (4)$$

where $\mathbf{A}_j \in G^{\mu \times \ell\mu}$ for each $j \in [\ell + 1]$ where for each $i \in [\mu]$ where each entry is zero except for having $g_{i,j}$ at the (i, ind_j^i) -th position. As in proof of Theorem 4, once such an equation does not hold, $\mathcal{B}$ computes g as follows:

Case 1: Inconsistency at the first entry of Eq. (4). In this case, for some $J \in [\ell + 1], I \in [\mu]$, we have $g_{J,I} \in G$ such that $g_{J,I} \star x = z_{J,I}$ from the representation of $z_{J,I}$, and $s_{J,I} \in G$ such that $s_{J,I}^{-1} \star \text{pk} = z_{J,I}$ from the forgery. We can obtain the desired element in this case.

Case 2: Inconsistency at the second entry of Eq. (4). The case is similar to the previous one. In this case, for some $J \in [\ell + 1], I \in [\mu]$, we have $g_{J,I} \in G$ such that $g_{J,I} \star \text{pk} = z_{J,I}$ from the representation of $z_{J,I}$, and $s_{J,I} \in G$ such that $s_{J,I} \star x = z_{J,I}$ from the forgery. We can obtain the desired element in this case.

Case 3: The rest of the cases. Since Eq. (4) does not hold for some $I \in [\ell + 1]$, say the set element by $c'_{J,I} \neq (\text{H}(\overrightarrow{\text{com}}_J \| m_J))_I$ for some $I \in [\mu]$. From the oracle query for the input $z_{J,I}$, we have an element g' such that $g'^{-1} \star z_{J,I}$ or $g' \star z_{J,I}$ is either x or pk . From the forgery, we have $s_{J,I} \star z_{J,I}$ or $s_{J,I}^{-1} \star z_{J,I}$ is either pk or x . We can obtain the desired group element.

It suffices to show Eq. (4) holds with a negligible chance over the random oracle model. To this end, we define the following three events.

- We define $\mathbf{E}_1$ as the event where $\exists j \in [\mu]$ such that the query $(z_1^j, \dots, z_\mu^j \| m_j)$ has never been made to H by $\mathcal{A}$.

- We define E_2 as the event where for some $j \in [\mu]$, the query $(z_1^j, \dots, z_\mu^j \| m_j)$ is first made during the gap when no sessions are open.
- We define E_3 as the event where for some $j \in [\mu]$, the query $(z_1^j, \dots, z_\mu^j \| m_j)$ is first made after the beginning of a session and before its closure.

Success conditioned on E_1 . Assuming there does not exist $s_{j,i} \in G$ in the forgeries such that both $s_{j,i} \star x = z_{j,i}$ and $s_{j,i} \star z_{j,i} = \text{pk}$ hold (as this would provide a GAIP solution between x and pk). Hence at most one equality out of two in the verification holds with the given forgeries. Modeling H as a random oracle, for each $j \in \mu$ the event E_1 happens with a probability of $\frac{1}{\binom{\mu}{\mu/2}}$. By the union bound, the probability is bounded above by $\frac{\ell+1}{\binom{\mu}{\mu/2}}$.

Success conditioned on $E_2 - E_1$. If the event occurs right before the beginning of the J -th session for some $J \in [\ell+1]$, the corresponding $c'_{j,i}$ values have been determined for every $j \in [J-1]$ and $i \in [\mu]$. The probability that such a query results in a forgery such that Eq. (4) holds is less than $\frac{1}{\binom{\mu}{\mu/2}}$. This result holds the same if the query is made after the last session. By the union bound, the probability is bounded above by $\frac{Q}{\binom{\mu}{\mu/2}}$.

Success conditioned on $E_3 - E_1 - E_2$. Since there are ℓ sessions with $\ell+1$ hash queries in the forgeries, there are at least two such queries made within the same session. For any two queries made within the J -th session, the corresponding $c'_{j,i}$ values in Eq. (4) have been determined for every $j \in [J-1]$ and $i \in [\mu]$. It follows that the corresponding two matrices of the two queries result in at most $\binom{\mu}{\mu/2}$ vectors over $\{0, 1\}^{2\mu}$, came from the undetermined $c'_{J,i}$ for $i \in [\mu]$. Hence, Eq. (4) holds with probability less than $\frac{\binom{\mu}{\mu/2}}{\binom{\mu}{\mu/2}^2}$. By the union bound, the probability is bounded by $\frac{\binom{Q}{2}}{\binom{\mu}{\mu/2}}$. Hence, we have $\text{Adv}_{\mathcal{A}, \ell}^{\text{SOMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + \frac{Q+\ell+1}{\binom{\mu}{\mu/2}} + \frac{\binom{Q}{2}}{\binom{\mu}{\mu/2}}$.

4 Second Framework: Multi-Key Blind Signature

In this section, we present another three-move blind signature scheme. We will prove it is sequential secure in the random oracle and algebraic group action model, assuming OMVP is hard. This construction can be seen as a generalization to multiple keys of Framework 1. This approach leads us to a trade-off between the number of keys and parallel repetitions (see Section 6), providing shorter signatures up to increasing the public key size. We describe the protocol in Framework 2 and refer to Appendix E for formal proofs of correctness and sequential one-more unforgeability. Here, we only sketch the proof idea.

We can place our scheme in the framework of *Schnorr-type blind signatures* as in [52]. It is based on the sigma protocol described in Fig. 4. It is easy to see that it satisfies correctness, special soundness, and honest verifier zero-knowledge properties. Indeed, given two transcripts (y, c_1, s_1) and (y, c_2, s_2) with same commitments and different challenges, we can extract $g \in G$ such that $g \star \text{pk}_{c_1} = \text{pk}_{c_2}$.

Prover(pk, sk)		Verifier(pk)
$h \leftarrow \$ G, y \leftarrow h \star x$	$\xrightarrow{y}$	
	$\xleftarrow{c}$	$c \leftarrow \$ [\kappa]$
$s \leftarrow h \cdot \text{sk}_c^{-1}$	$\xrightarrow{s}$	Check if $s \star \text{pk}_c = y$

Fig. 4. Basic Sigma protocol underlying Framework 2 with $\text{sk} = (g_1, \dots, g_\kappa)$ and $\text{pk} = (g_1 \star x, \dots, g_\kappa \star x)$.

Moreover, we can simulate a valid transcript of an interaction: sample a challenge c and group element h and return $(h \star \text{pk}_c, c, h)$. We call such a transcript a *simulated transcript*.

Let $(x, X, G, \star)$ be a cryptographic group action. Let α be a positive integer and $\kappa = \kappa(\lambda) \geq 2$, $\mu = \mu(\lambda) = \alpha \cdot \kappa(\lambda)$ be polynomial functions in the security parameter λ . Let $H : \{0, 1\}^* \rightarrow [\kappa]^\mu$ be a hash function with the following property: for any string m and $i, j \in [\kappa]$ the number of occurrences of i in $H(m)$ is equal to those of j .

KeyGen(λ): set $g_1 = id$ and sample group elements $g_2, \dots, g_\kappa \leftarrow \$ G^{\kappa-1}$. Output $\text{sk} = (g_2, \dots, g_\kappa)$ and $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa) = (x, g_2 \star x, \dots, g_\kappa \star x)$.

Signer₁(sk): sample group elements $h_1, \dots, h_\mu \leftarrow \$ G^\mu$. Compute $y_i = h_i \star x$ for each $i \in [\mu]$. Send $Y = (y_1, \dots, y_\mu)$ to User and keep $h_1, \dots, h_\mu$.

User₁(pk, Y, m): sample group elements $(r_1, \dots, r_\mu) \leftarrow \$ G^\mu$ and a random permutation $\pi : [\mu] \rightarrow [\mu]$. Compute $z_i = r_i \star y_i$ for all $i \in [\mu]$. Compute the challenge $C = H(z_{\pi(1)}, \dots, z_{\pi(\mu)}, m)$. Interpret C as a list $(c_1, \dots, c_\mu)$ and set $C' = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\mu)})$. Send C' to Signer.

Signer₂($\text{sk}, C', (h_1, \dots, h_\mu)$): Parse C' as $(c'_1, \dots, c'_\mu)$. For each $i \in [\mu]$ set $s'_i = h_i \cdot g_{c'_i}^{-1}$. Send $S' = (s'_1, \dots, s'_\mu)$ to the User.

User₂($\text{pk}, S', \pi, (r_1, z_1, \dots, r_\mu, z_\mu), m$): For each $i \in [\mu]$ set $s_i = r_i \cdot s'_i$. Output signature $\text{sig} = ((z_{\pi(1)}, \dots, z_{\pi(\mu)}), (s_{\pi(1)}, \dots, s_{\pi(\mu)}))$. Abort if it is invalid.

Verify(pk, m, sig): Parse $\text{sig} = ((z_1, \dots, z_\mu), (s_1, \dots, s_\mu))$ and compute the challenge $(c_1, \dots, c_\mu) = H(z_1, \dots, z_\mu, m)$. Output 1 if for all $i \in [\mu]$ the following holds: $s_i \star \text{pk}_{c_i} = z_i$. Otherwise, output 0.

Framework 2: Multi-key Blind Signature

4.1 Security Proofs

Theorem 6. *The blind signature presented in Framework 2 has perfect blindness.*

Proof. We show that any signature can be linked to any transcript of an execution of BS. In particular, we show that for any pair message/signature m, sig and any transcript $(y_1, \dots, y_\mu), (c'_1, \dots, c'_\mu)$ and $(s'_1, \dots, s'_\mu)$ of an execution of BS, there exist a permutation π and group elements $(r_1, \dots, r_\mu)$ that

$$\text{sig} = ((r_{\pi(1)} \star y_{\pi(1)}, \dots, r_{\pi(\mu)} \star y_{\pi(\mu)}), (r_{\pi(1)} \cdot s'_{\pi(1)}, \dots, r_{\pi(\mu)} \cdot s'_{\pi(\mu)})).$$

Let $\text{sig} = ((z_1, \dots, z_\mu), (s_1, \dots, s_\mu))$ be the signature. We compute $(c_1, \dots, c_\mu) = H((z_i), m)$ and compare it with $(c'_1, \dots, c'_\mu)$. By the property of H , there exists a permutation π such that $(c'_1, \dots, c'_\mu) = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\mu)})$. We now compute $(r_1, \dots, r_\mu)$ from the equalities $s_i = r_{\pi(i)} \cdot s'_{\pi(i)}$. We note that the permutation π is not unique. This is not a problem for this proof since the number of possible permutations is independent of the signature and the transcript.

Theorem 7. *Let BS be Framework 2. For any p.p.t. algebraic adversary $\mathcal{A}$ we have that $\text{Adv}_{\mathcal{A}, \ell}^{\text{SOMUF}}(\lambda)$ is negligible. More precisely, for any such $\mathcal{A}$, that makes at most $Q = Q(\lambda)$ queries, there exists a p.p.t. adversary $\mathcal{B}$ and a negligible function ε such that*

$$\text{Adv}_{\mathcal{A}, \ell}^{\text{SOMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + Q\varepsilon + \binom{Q}{2}\varepsilon,$$

where $\varepsilon = \binom{\mu}{\alpha, \dots, \alpha}^{-1} = \frac{\alpha!^\kappa}{\mu!}$.

Proof. Before starting with the proof let us fix some notation. Let $\mathcal{A}$ be an adversary as in the statement. We assume that $\mathcal{A}$ makes, at most, a polynomial number of queries $Q = Q(\lambda)$ to the random oracle H . We denote by $(m_1, \text{sig}_1), \dots, (m_{\ell+1}, \text{sig}_{\ell+1})$ its output, on input the public key $\text{pk} = (\text{pk}_0, \dots, \text{pk}_\kappa)$. More precisely, for $j \in [\ell + 1]$:

$$\text{sig}_j = ((z_{1,j}, \dots, z_{\mu,j}), (s_{1,j}, \dots, s_{\mu,j})).$$

Since the adversary is algebraic, it will also provide a description of each set element $z_{i,j}$ in terms of the previously seen set element. That is, for all i, j it will give us $\alpha_{i,j} \in G$ and $x_{i,j} \in X$, where $x_{i,j}$ is a set element that $\mathcal{A}$ saw during its execution, such that $z_{i,j} = \alpha_{i,j} \star x_{i,j}$. We will also assume that all the forgeries output by $\mathcal{A}$ are valid signatures, that is for all i, j

$$s_{i,j} \star \text{pk}_{c_{i,j}} = z_{i,j} \tag{5}$$

where for $c_{i,j}$ is defined as follows:

$$(c_{1,j}, \dots, c_{\mu,j}) = H(z_{1,j}, \dots, z_{\mu,j}, m_j).$$

For $J \in [\ell + 1]$ we define A_J and B_J as follows. A_J is the set of commitments $z_{i,J}$ that are simulated, that is the triplet $(z_{i,J}, c_{i,J}, s_{i,J})$ is a simulated transcript of the underlined sigma protocol. The set A_J is well defined. Since the adversary is algebraic, we say $z_{i,J}$ is *simulated* if and only if $z_{i,J} = s_{i,J} \star \text{pk}_{c_{i,J}}$. We define B_J as the set of commitments $z_{i,J} = \alpha_{i,J} \star x_{i,J}$ where $x_{i,j}$ is a signer commitment from a closed session (at the moment of the corresponding call to H) with the same challenge public key: namely, $c_{i,J}$ coincide with the challenge asked to the signer for $x_{i,J}$. We define $X_J = A_J \cup B_J$.

For $J \in [\ell + 1]$ let us denote with Open_J the set of indices corresponding to the sessions that are open when $\mathcal{A}$ queries the J -th forgery. We note that, by hypothesis on $\mathcal{A}$, $|\text{Open}_J| \leq 1$. For $I \in \text{Open}_J$ we define the set Ω_J^I as the collection of commitments in sig_J of the form $h \star y$ where y is a signer's commitment of session I , whose corresponding challenge matches the corresponding signer's response of session I (they are both with respect to the same public key).

Now we consider the following hybrid.

Game G_0 : The sequential one more unforgeability experiment $\text{sOMUF}(1^\lambda)$.

Game G_1 : This is G_0 with the following modification. When the adversary $\mathcal{A}$ outputs the signatures $(m_1, \text{sig}_1), \dots, (m_{\ell+1}, \text{sig}_{\ell+1})$, if there is a J such that $\mathcal{A}$ never queries $\mathsf{H}((z_{i,J}, m_J))$ we emulate a query via lazy sample. Then, we abort the experiment if at least one of the following two conditions is satisfied.

1. There exists $J \in [\ell + 1]$ such that $|X_J| = \mu$.
2. There exist $J_1, J_2 \in [\ell + 1]$ and $I \in [\ell]$ such that $|X_{J_1} \cup \Omega_{J_1}^I| = |X_{J_2} \cup \Omega_{J_2}^I| = \mu$.

Lemma 1. *Let E be the event “ $\mathcal{A}$ aborts in G_1 ”. Then $\Pr[E]$ is negligible.*

Proof. For a positive integer $T \leq \mu$ we denote with p_T the probability that for a given commitment $\overrightarrow{\text{com}}$, message m and challenges $c_1, \dots, c_T$ we have $\mathsf{H}(\overrightarrow{\text{com}}, m) = (c_1, \dots, c_T, *, \dots, *)$. Let E_1, E_2 be the events corresponding to the above abort conditions.

We have $\Pr[E] \leq \Pr[E_1] + \Pr[E_2]$. Let us first evaluate $\Pr[E_1]$. For $J \in [\ell + 1]$, we denote with Q_J the number of queries to H between the $(J - 1)$ -th and J -th forgery. We have that

$$\Pr[E_1] \leq \sum_{J \in [\ell + 1]} \Pr[|X_J| = \mu] = \sum_{J \in [\ell + 1]} Q_J \cdot p_\mu = Q \cdot \binom{\mu}{\alpha, \dots, \alpha}^{-1}$$

which is negligible by Lemma 4.

Now let us evaluate $\Pr[E_2]$. For $I \in [\ell]$ let Q_I be the number of queries to H made during session I . Let us denote with T_J the cardinality of X_J . We have

that:

$$\begin{aligned}
\Pr[E_2] &\leq \sum_{I \in [\ell]} \Pr[\exists J_1, J_2 \mid |X_{J_1} \cup \Omega_{J_1}^I| = |X_{J_2} \cup \Omega_{J_2}^I| = \mu] \\
&\leq \sum_{I \in [\ell]} \binom{Q_I}{2} \max_{T_{J_1}} \{p_{T_{J_1}} \cdot p_{\mu - T_{J_1}}\} \leq \sum_{I \in [\ell]} \binom{Q_I}{2} p_\mu \\
&\leq \binom{Q}{2} p_\mu = \binom{Q}{2} \binom{\mu}{\alpha, \dots, \alpha}^{-1}
\end{aligned}$$

which again is negligible by Lemma 4. We conclude that

$$\Pr[E] \leq \Pr[E_1] + \Pr[E_2] \leq (Q + \binom{Q}{2}) \binom{\mu}{\alpha, \dots, \alpha}^{-1}$$

is negligible.

We now proceed with the reduction to the *one-more vectorization problem*. Let $\mathcal{A}$ be a p.p.t. adversary playing G_1 . We construct an adversary $\mathcal{B}$ playing $G = \text{OMVP}$ as follows. Let $(x, X, G, \star)$ be the group action given in G .

1. Set $Y \leftarrow \text{Chall}()$.
2. Sample a random index $i^* \leftarrow [\kappa] \setminus \{1\}$. Set $g_1 = \text{id}$. For $i = 2, \dots, \kappa$ do the following. If $i = i^*$ set $\text{pk}_i = Y$, otherwise sample a group element $g_i \leftarrow G$ and set $\text{pk}_i = g_i \star x$.
3. Run $\mathcal{A}$ on input $(x, X, G, \star)$ and $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$.
4. Simulate the queries to H in the standard way and the queries to $\text{Sign}_1()$ and $\text{Sign}_2()$ as follows: let j denote the corresponding session,
 - Sign_1 : for $i \in [\mu]$ set $y_{i,j} \leftarrow \text{Chall}()$ and respond with $(y_{1,j}, \dots, y_{\mu,j})$.
 - Sign_2 : on input the challenge $(c_{1,j}, \dots, c_{\mu,j})$, for $i \in [\mu]$ set $s'_{i,j} \leftarrow \text{dLog}(\text{pk}_{c_{i,j}}, y_{i,j})$ and send $(s'_{1,j}, \dots, s'_{\mu,j})$. This simulation is correct since, $s'_{i,j} \star \text{pk}_{c_{i,j}} = y_{i,j}$ for all $i \in [\mu]$.
5. We receive $\ell + 1$ forgeries from $\mathcal{A}$

$$\begin{aligned}
m_1, \text{sig}_1 &= ((z_{1,1}, \dots, z_{\mu,1}), (s_{1,1}, \dots, s_{\mu,1})) \\
&\vdots \\
m_{\ell+1}, \text{sig}_{\ell+1} &= ((z_{1,\ell+1}, \dots, z_{\mu,\ell+1}), (s_{1,\ell+1}, \dots, s_{\mu,\ell+1}))
\end{aligned}$$

together with $\alpha_{i,j} \in G$ and $x_{i,j} \in X$ such that $z_{i,j} = \alpha_{i,j} \star x_{i,j}$, where $x_{i,j}$ is a set element saw during its run.

6. Our goal is to extract, from the forgeries, $h \in G$ such that $h \star x = Y$. Depending on the queries made by $\mathcal{A}$ to H we distinguish two cases.
Case 1. There exists $J \in [\ell + 1]$ such that $\mathcal{A}$ never queries $H((z_{i,j}), m_J)$. Then, there exists $I \in [\mu]$ such that $z_{I,J} \notin X_J$ (since $|X_J| < \mu$ by the first abort condition). If $z_{I,J} = \alpha_{I,J} \star x$ then, by Equation 11, we have

$$s_{I,J} \star \text{pk}_{c_{I,J}} = z_{I,J} = \alpha_{I,J} \star x,$$

Therefore, if $c_{I,J} = i^*$, we have $s_{I,J} \star Y = \alpha_{I,J} \star x$ and hence we have $h = s_{I,J}^{-1} \cdot \alpha_{I,J}$. Since H is a random oracle, $\Pr[c_{I,J} = i^*] = 1/\kappa$.

If there exists $i \in [\mu]$ and $j \in [\ell]$ such $z_{I,J} = \alpha_{I,J} \star y_{i,j}$ and $c_{i,j} \neq c_{I,J}$ or (because $z_{I,J} \notin X_J$). On one hand we have $y_{i,j} = s_{i,j} \star \text{pk}_{c_{i,j}}$, on the other, applying Equation 11, $y_{i,j} = \alpha_{I,J}^{-1} \star z_{I,J} = (\alpha_{I,J}^{-1} \cdot s_{I,J}) \star \text{pk}_{c_{I,J}}$. We thus obtain the relation

$$s_{i,j} \star \text{pk}_{c_{i,j}} = (\alpha_{I,J}^{-1} \cdot s_{I,J}) \star \text{pk}_{c_{I,J}}$$

with $c_{i,j} \neq c_{I,J}$. If $c_{i,j} = i^*$ we are able to extract h . Since H is a random oracle $\Pr[c_{i,j} = i^*] = 1/\kappa$. Otherwise, $z_{I,J} = \alpha_{I,J} \star \text{pk}_c$ for some c . Then $c_{I,J} \neq c$ (because the commitment is not simulated), and we can extract similarly.

Case 2. For all $J \in [\ell+1]$ the adversary queries $H((z_{i,J}), m_J)$. We distinguish two subcases.

- Exists a query that was made in between two sessions, that means before a i -th Sign_1 query and after the $(i-1)$ -th Sign_2 call. This case is analogous to the previous one and the probability of extraction is also $1/\kappa$.
- All the queries for the forgeries are done between some i -th Sign_1 and i -th Sign_2 call. It follows that there exists $I \in [\ell]$ and $J \in [\ell+1]$ such that $\mathcal{A}$ asks $H((z_{i,J}), m_J)$ and $H((z_{i,J+1}), m_{J+1})$ between the I -th Sign_1 and the I -th Sign_2 . Because we are in a sequential setting, all the i -th sessions for $i < I$ are closed when $\mathcal{A}$ queries for those forgeries. By the second abort condition in G_1 , there exists a commitment z in those forgeries of the form $z = \alpha \star x$ or $z = \alpha \star y$ for some y from the i -th Sign_1 commitments for $i \leq I$ and with different challenges. Then we can extract h with probability $1/\kappa$, as in case 1.

We have shown that from the forgeries returned by $\mathcal{A}$ we can extract h such that $Y = h \star x$ with non-negligible probability $1/\kappa$. It is easy to see that the knowledge of h allows us to recover all the other challenges $h_{i,j}$ such that $y_{i,j} = h_{i,j} \star x$. Therefore, up to re-run $\mathcal{A}$ a polynomial number of times, we let $\mathcal{B}$ respond to OMVP challenger with $(h, (h_{i,j}))$. It follows that

$$\text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) = \text{Adv}_{\mathcal{A}}^{G_1}(\lambda).$$

In conclusion,

$$\text{Adv}_{\mathcal{A},\ell}^{\text{sOMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + \Pr[E] \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + Q\varepsilon + \binom{Q}{2}\varepsilon$$

where $\varepsilon = \left(\frac{\mu}{\alpha, \dots, \alpha}\right)^{-1}$.

5 Third Framework: Randomized Multi-Key Variant

Let KeyGen be the key generation function of the previous scheme. We introduce the algorithm KeyRand : on input a public key $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$ sample a vector

$(t_1, \dots, t_\kappa) \leftarrow \$ G^\kappa$ and permutation $\sigma \leftarrow \$ S_\kappa$, randomize $\mathbf{pk}$ by computing $\mathbf{pk}' = (t_{\sigma(1)} \star \mathbf{pk}_{\sigma(1)}, \dots, t_{\sigma(\kappa)} \star \mathbf{pk}_{\sigma(\kappa)})$ and output $(\mathbf{pk}', (t_1, \dots, t_\kappa), \sigma)$. We describe the construction in Framework 3 and discuss blindness and one-more unforgeability in the following sections.

5.1 Security Proofs

Theorem 8. *The blind signature presented in Framework 3 is correct.*

Proof. Let $\mathbf{sig} = (Z, S, T, \tau)$ be the output of User_2 , we show that it is a valid signature: namely that $\text{Verify}(\mathbf{pk}, m, \mathbf{sig}) = 1$. We have

$$\mathbf{sig} = ((z_{\pi(1)}, \dots, z_{\pi(\lambda)}), (s_{\pi(1)}, \dots, s_{\pi(\lambda)}), (t_{\sigma(1)} \cdot t'_{\sigma'(\sigma(1))}, \dots, t_{\sigma(k)} \cdot t'_{\sigma'(\sigma(k))}), \sigma' \circ \sigma)$$

and

$$\overline{\mathbf{pk}} = (t_{\sigma(1)} \cdot t'_{\sigma'(\sigma(1))} \star \mathbf{pk}_{\sigma'(\sigma(1))}, \dots, t_{\sigma(k)} \cdot t'_{\sigma'(\sigma(k))} \star \mathbf{pk}_{\sigma'(\sigma(k))}) = \mathbf{pk}''. \quad (6)$$

Therefore $H(Z, m, \mathbf{pk}') = H(z_{\pi(1)}, \dots, z_{\pi(\lambda)}, \mathbf{pk}'', m) = (c_1, \dots, c_\lambda)$. We show that for $i = 1, \dots, \lambda$

$$s_{\pi(i)} \star \mathbf{pk}_{c_i}'' = z_{\pi(i)}.$$

From the protocol we have

$$\begin{aligned} s_{\pi(i)} &= r_{\pi(i)} \cdot s'_{\pi(i)} \cdot t_{c'_{\pi(i)}}^{-1} \\ &= r_{\pi(i)} \cdot h_{\pi(i)} \cdot g_{c'_{\pi(i)}}'^{-1} \cdot t_{c'_{\pi(i)}}^{-1} \\ &= r_{\pi(i)} \cdot h_{\pi(i)} \cdot g_{\sigma(c_i)}'^{-1} \cdot t_{\sigma(c_i)}^{-1} \\ &= r_{\pi(i)} \cdot h_{\pi(i)} \cdot (t'_{\sigma'(\sigma(c_i))} \cdot g_{\sigma'(\sigma(c_i))})^{-1} \cdot t_{\sigma(c_i)}^{-1} \\ &= r_{\pi(i)} \cdot h_{\pi(i)} \cdot (t_{\sigma(c_i)} \cdot t'_{\sigma'(\sigma(c_i))} \cdot g_{\sigma'(\sigma(c_i))})^{-1}. \end{aligned}$$

From Equation 6 it follows that

$$\mathbf{pk}_{c_i}'' = (t_{\sigma(1)} \cdot t'_{\sigma'(\sigma(1))} \cdot g_{\sigma'(\sigma(1))}) \star x.$$

In conclusion

$$s_{\pi(i)} \star \mathbf{pk}_{c_i}'' = r_{\pi(i)} \cdot h_{\pi(i)} \star x = z_{\pi(i)}.$$

Theorem 9. *The blind signature presented in Framework 3 has perfect blindness.*

Proof. We show that any signature can be linked to any transcript of a non-aborting execution of BS. In particular, we show that for any pair message/signature $m, \mathbf{sig}$ and any transcript $[(\mathbf{pk}', (y_1, \dots, y_\mu)), (c'_1, \dots, c'_\mu), ((s'_1, \dots, s'_\mu), t', \sigma')]$

of an execution of BS, there exist permutations π, σ and group elements $(r_1, \dots, r_\mu)$ and $(t_1, \dots, t_\kappa)$ such that

$$\begin{aligned} \mathbf{sig} = & ((r_{\pi(1)} \star y_{\pi(1)}, \dots, r_{\pi(\mu)} \star y_{\pi(\mu)}), \\ & (r_{\pi(1)} \cdot s'_{\pi(1)} \cdot t_{c'_{\pi(1)}}^{-1}, \dots, r_{\pi(\mu)} \cdot s'_{\pi(\mu)} \cdot t_{c'_{\pi(\mu)}}^{-1}), \\ & (t_{\sigma(1)} \cdot t'_{\sigma'(\sigma(1))}, \dots, t_{\sigma(\kappa)} \cdot t'_{\sigma'(\sigma(\kappa))}), \sigma' \circ \sigma). \end{aligned}$$

Let $\mathbf{sig} = ((z_1, \dots, z_\mu), (s_1, \dots, s_\mu), (\tilde{t}_1, \dots, \tilde{t}_\kappa), \tilde{\sigma})$ be the signature and $\widetilde{\mathbf{pk}}$ the corresponding public key. We compute $\sigma = (\sigma')^{-1} \circ \tilde{\sigma}$. From $\sigma, \sigma', (\tilde{t}_1, \dots, \tilde{t}_\kappa)$ and $(t'_1, \dots, t'_\kappa)$ we compute $(t_1, \dots, t_\kappa)$. We evaluate $(c_1, \dots, c_\mu) = H((z_i), \widetilde{\mathbf{pk}}, m)$ and compare it with $(c'_1, \dots, c'_\mu)$. By the property of H and the knowledge of σ , there exists a permutation π such that $(c'_1, \dots, c'_\mu) = (\sigma(c_{\pi^{-1}(1)}), \dots, \sigma(c_{\pi^{-1}(\mu)}))$. We recover $(r_1, \dots, r_\mu)$ from the following equalities $s_i = r_{\pi(i)} \cdot s'_{\pi(i)} \cdot t_{c'_{\pi(i)}}^{-1}$. We note that π is not unique. This is not a problem for the proof since the number of possible permutations is independent of the signature and the transcript.

Concurrent One-more Unforgeability.

Theorem 10. *Let BS be Framework 3. For any p.p.t. algebraic adversary $\mathcal{A}$ we have that $\text{Adv}_{\mathcal{A}, \ell}^{\text{COMUF}}(\lambda)$ is negligible. More precisely, for any such $\mathcal{A}$, that makes at most $Q = Q(\lambda)$ queries to H , there exists a p.p.t. adversary $\mathcal{B}$ and a negligible function $\varepsilon = \varepsilon(\lambda)$ such that*

$$\text{Adv}_{\mathcal{A}, \ell}^{\text{COMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + \varepsilon.$$

where $\varepsilon = Q \binom{\mu}{\alpha, \dots, \alpha}^{-1} + (\ell + 1) \frac{(\kappa/2)!}{\kappa!} + Q^2 p_{\mu/2} \frac{(\kappa/2)!}{\kappa!}$ and for $T \in [\mu]$, $p_T = \frac{\alpha!^\kappa}{\mu!} \cdot \frac{(\mu - \kappa \lfloor T/\kappa \rfloor)!}{(\alpha - \lfloor T/\kappa \rfloor)!^\kappa}$.

Proof. Let $\mathcal{A}$ be an adversary as in the statement. We assume that $\mathcal{A}$ makes, at most, a polynomial number of queries $Q = Q(\lambda)$. We denote by $(m_1, \mathbf{sig}_1), \dots, (m_{\ell+1}, \mathbf{sig}_{\ell+1})$ its output, on input the public key $\mathbf{pk} = (\mathbf{pk}_1, \dots, \mathbf{pk}_\kappa)$. More precisely, for $j \in [\ell + 1]$:

$$\mathbf{sig}_j = ((z_{1,j}, \dots, z_{\mu,j}), (s_{1,j}, \dots, s_{\mu,j}), (t_{1,j}, \dots, t_{\kappa,j}), \sigma_j).$$

Since the adversary is algebraic, it will also describe each set element $z_{i,j}$ in terms of the previously seen set element. That is, for all i, j it will give us $\alpha_{i,j} \in G$ and $x_{i,j} \in X$, where $x_{i,j}$ is a set element that $\mathcal{A}$ saw during its execution, such that $z_{i,j} = \alpha_{i,j} \star x_{i,j}$. We will also assume that all the forgeries output by $\mathcal{A}$ are valid signatures, that is for all i, j

$$s_{i,j} \star \overline{\mathbf{pk}}_{c_{i,j}} = z_{i,j} \tag{7}$$

where $c_{i,j}$ is defined as follows: $(c_{1,j}, \dots, c_{\mu,j}) = H(z_{1,j}, \dots, z_{\mu,j}, \overline{\mathbf{pk}}_j, m_j)$ where $\overline{\mathbf{pk}}_j$ is defined as $\overline{\mathbf{pk}}_j = (t_1 \star \mathbf{pk}_{\sigma(1)}, \dots, t_\kappa \star \mathbf{pk}_{\sigma(\kappa)})$.

Let $(x, X, G, \star)$ be a cryptographic group action. Let α be a positive integer and $\kappa = \kappa(\lambda) \geq 2$, $\mu = \mu(\lambda) = \alpha \cdot \kappa(\lambda)$ be polynomial functions in the security parameter λ . Let $H : \{0, 1\}^* \rightarrow [\kappa]^\mu$ be a hash function with the following property: for any string m and $i, j \in [\kappa]$ the number of occurrences of i in $H(m)$ is equal to those of j .

KeyGen(λ): set $g_1 = id$ and sample group elements $g_2, \dots, g_\kappa \leftarrow G^{\kappa-1}$. Output $sk = (g_2, \dots, g_\kappa)$ and $pk = (pk_1, \dots, pk_\kappa) = (x, g_2 \star x, \dots, g_\kappa \star x)$.

Signer₁(sk, pk): compute $pk', t', \sigma' \leftarrow \text{KeyRand}(pk)$. For $i \in [\kappa]$ set $g'_i = t'_{\sigma'(i)} \cdot g_{\sigma'(i)}$. Sample group elements $h_1, \dots, h_\mu \leftarrow G$ and compute $y_i = h_i \star x$ for each $i \in [\mu]$. Send $pk', (y_1, \dots, y_\mu)$ to User and keep $h_1, \dots, h_\mu, g'_1, \dots, g'_\kappa, t', \sigma'$.

User₁($pk', y_1, \dots, y_\mu, m$): compute $pk'', t, \sigma \leftarrow \text{KeyRand}(pk')$. Sample group elements $r_1, \dots, r_\mu \leftarrow G$ and random permutation π . For $i \in [\mu]$ compute $z_i = r_i \star y_i$. Compute the challenge $c = H(z_{\pi(1)}, \dots, z_{\pi(\mu)}, pk'', m)$. Interpret c as a list $c_1, \dots, c_\mu$ and set $c' = (\sigma(c_{\pi^{-1}(1)}), \dots, \sigma(c_{\pi^{-1}(\mu)}))$. Send c' to Signer.

Signer₂($c', g'_1, \dots, g'_\kappa, h_1, \dots, h_\mu, t', \sigma'$): parse c' as $(c'_1, \dots, c'_\mu)$. For each $i \in [\mu]$ set $s'_i = h_i \cdot (g'_{c'_i})^{-1}$. Send $S' = (s'_1, \dots, s'_\mu, t', \sigma')$ to the User.

User₂($pk, S', \pi, (r_1, z_1, \dots, r_\mu, z_\mu), m$): Verify $pk' = \text{KeyGen}(pk; t', \sigma')$ holds and verify S' . For each $i \in [\mu]$ set $s_i = r_i \cdot s'_i \cdot t_{c'_i}^{-1}$. Output signature sig if it is valid, where $\text{sig} = ((z_{\pi(1)}, \dots, z_{\pi(\mu)}), (s_{\pi(1)}, \dots, s_{\pi(\mu)}), (t_{\sigma(1)} \cdot t'_{\sigma'(\sigma(1))}, \dots, t_{\sigma(\kappa)} \cdot t'_{\sigma'(\sigma(\kappa))}), \sigma' \circ \sigma)$.

Verify(pk, m, sig): Parse $\text{sig} = ((z_1, \dots, z_\mu), (s_1, \dots, s_\mu), (t_1, \dots, t_\kappa), \sigma)$. Set $\overline{pk} = (t_1 \star pk_{\sigma(1)}, \dots, t_\kappa \star pk_{\sigma(\kappa)})$ and compute $(c_1, \dots, c_\mu) = H(z_1, \dots, z_\mu, \overline{pk}, m)$. Output 1 if for any $i \in [\mu]$ the following holds: $s_i \star \overline{pk}_{c_i} = z_i$. Otherwise, output 0.

Framework 3: Randomized Multi-key Blind Signature

For $J \in [\ell + 1]$ we define A_J and B_J as follows. A_J is the set of commitments $z_{i,J}$ that are *simulated*, that is the triplet of elements (from the signature) $(z_{i,J}, c_{i,J}, s_{i,J})$ is a *simulated* transcript of the underlined sigma protocol, which means $z_{i,J} = s_{i,J} \star \overline{pk}_{c_{i,J}}$. The set A_J is well defined. We define B_J as the set of commitments $z_{i,J} = \alpha_{i,J} \star x_{i,J}$ where $x_{i,J}$ is a signer commitment from a closed session (at the moment of the corresponding call to H) with the same challenge public key: namely, $c_{i,J}$ coincides with the challenge forwarded from **User₁** to the signer for $x_{i,J}$ for the same session-specific (ephemeral) key. We define $X_J = A_J \cup B_J$. Roughly speaking, in this event adversary is guessing with respect to the hash queries.

For $J \in [\ell + 1]$ let us denote with Open_J the set of indices corresponding to the sessions that are open when $\mathcal{A}$ is querying the hash of the J -th forgery. For $I \in \text{Open}_J$ we define the set Ω_J^I as the collection of commitments in sig_J of the form $h \star y$ where y is a signer's commitment of session I , whose corresponding challenge matches the corresponding signer's response of session I (they are both linked to the same public key in the end). Roughly speaking, in this event

adversary is guessing with respect to the signer's hidden randomness generating the ephemeral key. Now let us consider the following games.

Game G_1 : This is $G_0 := \text{cOMUF}(1^\lambda)$ with the following modification. When the adversary $\mathcal{A}$ outputs the signatures $(m_1, \text{sig}_1), \dots, (m_{\ell+1}, \text{sig}_{\ell+1})$, if there is a J such that $\mathcal{A}$ never queries $H((z_{i,J}, -, m_J))$ we emulate a query via lazy sample, otherwise we do nothing. Then, we abort the experiment if at least one of the three conditions is satisfied.

1. There exists $J \in [\ell + 1]$ such that $|X_J| \geq \mu$.
2. There exists $J \in [\ell + 1]$ such that for all $I \in \text{Open}_J$, $|\bigcup_{i \neq I} \Omega_J^i| \geq \mu/2$.
3. There exist $J_1, J_2 \in [\ell + 1]$ such that there exists $I \in \text{Open}_{J_1} \cap \text{Open}_{J_2} \subseteq [\ell]$ such that $|X_{J_1} \cup \Omega_{J_1}^I \cup \bigcup_{i \neq I} \Omega_{J_1}^i| = |X_{J_2} \cup \Omega_{J_2}^I \cup \bigcup_{i \neq I} \Omega_{J_2}^i| = \mu$ and $|X_{J_1} \cup \Omega_{J_1}^I|, |X_{J_2} \cup \Omega_{J_2}^I| \geq \mu/2$.

Lemma 2. *Let E be the event “ $\mathcal{A}$ aborts in G_1 ”. Then $\Pr[E]$ is negligible.*

Proof. Let $Q = Q(\lambda)$ be the number of queries $\mathcal{A}$ makes to the random oracle. Let E_1, E_2, E_3 be the events corresponding to the above abort conditions. We have that $\Pr[E] \leq \Pr[E_1] + \Pr[E_2] + \Pr[E_3]$. We evaluate each probability separately. For the first one, we have that

$$\Pr[E_1] \leq Q \cdot \binom{\mu}{\alpha, \dots, \alpha}^{-1}$$

is negligible. We refer to lemma 8 in the appendix for a proof of this statement.

We next show that $\Pr[E_2]$ is negligible. The intuition here is the following. Since the attacker doesn't know the I -th signer's permutation (applied to the public key) until they query Sign_2 (and hence close the session), and since the final public key is part of the hash, each session is independent from the others. We want to evaluate $\Pr[E_1] = \Pr[\exists J \mid \forall I, |\bigcup_{i \neq I} \Omega_J^i| \geq \mu/2]$. For a fixed J we estimate $\Pr[E_J] = \Pr[\forall I, |\bigcup_{i \neq I} \Omega_J^i| \geq \mu/2]$. Let I_0 be such that $|\bigcup_{i \neq I_0} \Omega_J^i|$ is minimum. To abort, this cardinality must be greater or equal to $\mu/2$. Moreover, the challenges corresponding to the Ω_J^i for $i \neq I_0$, must agree with the unknown i -th signer's permutation. Therefore, we have $\Pr[E_J] \leq \prod_{i \neq I_0} \frac{1}{\kappa(\kappa-1) \cdots (\kappa - \delta_J^i + 1)}$. Where δ_J^i is defined as the number of different challenges among those corresponding to the commitments in Ω_J^i . To bound this probability, we consider the best case (for the adversary) when $\text{Open}_J = \{I_0, I_1\}$ and $\delta_J^{I_1} = \kappa/2$. Hence we have

$$\prod_{i \neq I_0} \frac{1}{\kappa(\kappa-1) \cdots (\kappa - \delta_J^i + 1)} \leq \frac{1}{\kappa(\kappa-1) \cdots (\kappa - \kappa/2 + 1)} = \frac{(\kappa/2)!}{\kappa!}.$$

Using the Stirling approximation 3 we have that $(\kappa/2)!/\kappa! \approx \sqrt{1/2}(e/2\kappa)^{\kappa/2}$ is negligible. In conclusion:

$$\Pr[E_2] \leq \sum_J \Pr[E_J] \leq (\ell + 1) \frac{(\kappa/2)!}{\kappa!}$$

is negligible.

We conclude by showing that $\Pr[E_3]$ is negligible. First, we observe that for a given commitment $\overrightarrow{\text{com}}$, set elements $\overline{\text{pk}}$, message m and challenges $c_1, \dots, c_T$ where $\text{H}(\overrightarrow{\text{com}}, \overline{\text{pk}}, m)$ has never been queried before, we have $\text{H}(\overrightarrow{\text{com}}, \overline{\text{pk}}, m)$ is of a given form $(c_1, \dots, c_T, *, \dots, *)$ with the probability

$$p_T \leq \frac{\alpha!^\kappa}{\mu!} \cdot \frac{(\mu - \kappa \lfloor T/\kappa \rfloor)!}{(\alpha - \lfloor T/\kappa \rfloor)!^\kappa}.$$

For $I \in [\ell]$ let Q_I be the number of queries to H made during the I -th session. Let us denote with T_J the cardinality of X_J . Let us denote with E_I the event “ $\exists J_1, J_2$ such that $|X_{J_1} \cup \Omega_{J_1}^I \bigcup_{i \neq I} \Omega_{J_1}^i| = |X_{J_2} \cup \Omega_{J_2}^I \bigcup_{i \neq I} \Omega_{J_2}^i|$ and $|X_{J_1} \cup \Omega_{J_1}^I|, |X_{J_2} \cup \Omega_{J_2}^I| \geq \mu/2$ ”. We have that $\Pr[E_3] \leq \sum_{I \in [\ell]} \Pr[E_I]$. To estimate $\Pr[E_I]$ we can assume that $\Omega_{J_1}^I \cup \Omega_{J_2}^I$ consists of all the signer's commitments and therefore $|X_{J_1} \cup X_{J_2} \bigcup_{i \neq I} \Omega_{J_1}^i \bigcup_{i \neq I} \Omega_{J_2}^i| = \mu$. Finally, we have that

$$\begin{aligned} \Pr[E_3] &\leq \sum_{I \in [\ell]} \Pr[E_I] \\ &\leq \sum_{I \in [\ell]} \binom{Q_I}{2} \max_{T_{J_1}, T_{J_2}} \{p_{T_{J_1}} \cdot p_{T_{J_2}}\} \prod_{k \in \{1, 2\}} \prod_{i \neq I_0} \frac{1}{\kappa(\kappa - 1) \dots (\kappa - \delta_{J_k}^i + 1)} \\ &\leq \sum_{I \in [\ell]} \binom{Q_I}{2} p_{\mu/2} \frac{(\kappa/2)!}{\kappa!} \leq \binom{Q}{2} p_{\mu/2} \frac{(\kappa/2)!}{\kappa!} \end{aligned}$$

is negligible.

We now proceed with the reduction to OMVP. Let $\mathcal{A}$ be a p.p.t. adversary playing G_1 . We construct an adversary $\mathcal{B}$ playing $G = \text{OMVP}$ as follows. Let $(x, X, G, \star)$ be as in our BS.

1. Set $Y \leftarrow \text{Chall}()$. Sample index $i^* \leftarrow [\kappa] \setminus \{1\}$. Set $g_1 = id$. For $i = 2, \dots, \kappa$ do the following. If $i = i^*$ set $\text{pk}_i = Y$, otherwise sample $g_i \leftarrow G$ and set $\text{pk}_i = g_i \star x$.
2. Run $\mathcal{A}$ on input $(x, X, G, \star)$ and $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$.
3. Simulate the queries to H in the standard way and the queries to $\text{Sign}_1()$ and $\text{Sign}_2()$ as follows.
 - j -th Sign_1 : compute $\text{pk}'_j, t'_j, \sigma'_j \leftarrow \text{KeyRand}(\text{pk})$. For $i \in [\mu]$ set $y_{i,j} \leftarrow \text{Chall}()$ and respond with $(y_{1,j}, \dots, y_{\mu,j})$ and pk'_j .
 - j -th Sign_2 : on input the challenge $(c'_{1,j}, \dots, c'_{\mu,j})$, for $i \in [\mu]$ set $s'_{i,j} \leftarrow \text{dLog}(\text{pk}'_{c'_{i,j}}, y_{i,j})$ and return $(s'_{1,j}, \dots, s'_{\mu,j}), t'_j, \sigma'_j$. This simulation is correct since, $s'_{i,j} \star \text{pk}'_{c'_{i,j}} = y_{i,j}$ for all $i \in [\mu]$.
4. We receive the forgeries from $\mathcal{A}$

$$m_j, \text{sig}_j = ((z_{1,j}, \dots, z_{\mu,j}), (s_{1,j}, \dots, s_{\mu,j}), (t_{1,j}, \dots, t_{\kappa,j}), \sigma_j), \text{ for } j \in [\ell + 1].$$

together with $\alpha_{i,j} \in G$ and $x_{i,j} \in X$ such that $z_{i,j} = \alpha_{i,j} \star x_{i,j}$, where $x_{i,j}$ is a set element saw during its execution.

5. From the forgeries we extract $h \in G$ such that $h \star x = Y$ with probability $1/\kappa$. Depending on the queries made by $\mathcal{A}$ to $\mathbf{H}$ we distinguish two cases.
- Case 1. There exists $J \in [\ell + 1]$ such that $\mathcal{A}$ never queries $\mathbf{H}((z_{i,J}), \overline{\text{pk}}_J, m_J)$. Then, there exists $I \in [\mu]$ such that $z_{I,J} \notin X_J$ (since $|X_J| < \mu$ from the first abort condition). If $z_{I,J} = \alpha_{I,J} \star x$ then, by Equation 7, we have $s_{I,J} \star \overline{\text{pk}}_{c_{I,J}} = z_{I,J} = \alpha_{I,J} \star x$. Therefore, if $\overline{\text{pk}}_{c_{I,J}}$ corresponds to pk_{i^*} , we have $(s_{I,J} \cdot t_{\sigma(c_{I,J})}) \star Y = \alpha_{I,J} \star x$ and hence we have $h = (s_{I,J} \cdot t_{\sigma(c_{I,J})})^{-1} \cdot \alpha_{I,J}$. Since $\mathbf{H}$ is a random oracle, $\Pr[\sigma(c_{I,J}) = i^*] = 1/\kappa$. If there exists $i \in [\mu]$ and $j \in [\ell]$ such $z_{I,J} = \alpha_{I,J} \star y_{i,j}$ and $c'_{i,j} \neq c_{I,J}$ (because $z_{I,J} \notin X_J$). On one hand we have $y_{i,j} = s'_{i,j} \star \text{pk}'_{c'_{i,j}}$, on the other, applying Equation 7, $y_{i,j} = \alpha_{I,J}^{-1} \star z_{I,J} = (\alpha_{I,J}^{-1} \cdot s_{I,J}) \star \overline{\text{pk}}_{c_{I,J}}$. We thus obtain the relation $s'_{i,j} \star \text{pk}'_{c'_{i,j}} = (\alpha_{I,J}^{-1} \cdot s_{I,J}) \star \overline{\text{pk}}_{c_{I,J}}$ with $c'_{i,j} \neq c_{I,J}$. If $c'_{i,j} = i^*$ or $\sigma(c_{I,J}) = i^*$ we are able to extract h . Since $\mathbf{H}$ is a random oracle $\Pr[c'_{i,j} = i^*] = 1/\kappa$. Otherwise, $z_{I,J} = \alpha_{I,J} \star \text{pk}_c$ for some c . Then $\sigma(c_{I,J}) \neq c$ (because the commitment is not simulated), and we can extract.
- Case 2. For all $J \in [\ell + 1]$ the adversary queries $\mathbf{H}((z_{i,J}), \overline{\text{pk}}_J, m_J)$. We distinguish two subcases.
- Exists a query that was made in between two sessions, that means before a i -th Sign_1 query and after the previous $(i - 1)$ -th Sign_2 call. This case is analogous to the previous one and the probability of extraction is $1/\kappa$.
 - All the queries for the forgeries are done between some i -th Sign_1 and i -th Sign_2 call. If there is a J such that $|X_J \cup \bigcup_{i \in \text{Open}_J} \Omega_J^i| < \mu$ then either there exist commitments from which we can extract as in case 1, or among the $z_{i,J}$'s there must be one of the form $z_{i,j} = \alpha_{i,j} \star x$, or $z_{i,j} = \alpha_{i,j} \star y$ for a Sign_1 commitment with different challenge public key. Therefore in this case we can extract with probability $1/\kappa$ similarly to case 1. Otherwise, $|X_J \cup \bigcup_{i \in \text{Open}_J} \Omega_J^i| = \mu$ for all $J \in [\ell + 1]$. We show that this case cannot happen. Thanks to the first abort condition, for all J there exists I such that $|X_J \cup \Omega_J^I| > \mu/2$. Since the sessions are ℓ and the forgeries are $\ell + 1$ there exist $J_1 \neq J_2$ and I such that $|X_{J_1} \cup \Omega_{J_1}^I|$ and $|X_{J_2} \cup \Omega_{J_2}^I|$ are greater then $\mu/2$. And this contradicts the second abort condition.
6. Up to rerun $\mathcal{A}$ a polynomial number of times, we extract h . It is easy to see that from h we recover all the other challenge discrete logarithms.

It follows that $\text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{G}_1}(\lambda)$. In conclusion,

$$\text{Adv}_{\mathcal{A}, \ell}^{\text{COMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + 2Q\varepsilon$$

where ε is the negligible function from Lemma 2.

6 Instantiations

In this section, we show the performance of Frameworks 1 to 3 when instantiated with concrete post-quantum cryptographic group actions. We focus on two representative candidates: CSIDH-512 from isogeny-based cryptography and LESS

from code-based cryptography. For each, we describe how to compute the sizes of public keys and signatures and explain the choice of the parameters in Frameworks 1 to 3. For future work, we discuss some potential optimization of our instantiation in Remark 4 also discuss potential post-quantum candidates in Remark 3.

6.1 Post-quantum Group Actions

CSIDH/CSI-FiSh. The isogeny-based key exchange CSIDH [35] uses the action of the ideal class group G of the quadratic order $\mathbb{Z}[\sqrt{-p}]$ on the set X of supersingular elliptic curves defined over a prime field $\mathbb{F}_p$. The scheme fits into our frameworks using the variant, CSIFiSh (CSIDH-512), in [21]. To have an effective group action (i.e. the cryptographic group action considered in our work) has been an active line of research [49,38,47]. The elements of X can be represented as an element in $\mathbb{F}_p$ and the group element G can be represented as an element in $\mathbb{Z}/N\mathbb{Z}$ where $N \approx \sqrt{p}$. For our instantiation, we will refer to the reference implementation of CSI-FiSh (CSIDH-512) in [21].

LESS. LESS [23], a code-based signature scheme submitted to the NIST call, is based on the hardness of the linear code equivalence problem. This problem naturally fits the framework of cryptographic group actions. The set X consists of k -dimensional linear codes of length n over $\mathbb{F}_q$, represented by generator matrices in $\mathbb{F}_q^{k \times n}$. The group G is taken to be the monomial group $\text{Mon}_n(\mathbb{F}_q)$, consisting of matrices that are the product of a permutation matrix and a non-singular diagonal matrix. The group action is defined as:

$$\star: G \times X \rightarrow X, \quad (M, x) \mapsto \text{RREF}(xM),$$

where $\text{RREF}(\cdot)$ denotes the row-reduced echelon form. Although G is not commutative and the action is not transitive, these properties do not hinder its applicability in our constructions.

In LESS, public keys, elements in X , are chosen to be in systematic form, enabling storage as the $k \times (n - k)$ suffix of the generator matrix. For the parameter set $(n, k, q) = (252, 126, 127)$ recommended in [80, §6] for 128-bit security, this results in a public key size of 13.6 KB [80, Table 2]. Group elements in G can be represented using $n \lceil \log_2 n \rceil + n \lceil \log_2(q - 1) \rceil$ bits.

6.2 Performance

Size and Security Comparison. Table 1 provides a comparative overview of our schemes with existing sigma-protocol-based post-quantum blind signatures, wrt keys, signature, and provable security guarantees. The reasoning of the parameter choices is in Appendix F. Under the sequential security setting, instantiations of our Framework 2 give the most compact signature sizes among all known schemes. Furthermore, our Framework 3 represents the first concrete blind signature construction in both the isogeny-based and code-based literature that provably achieves security against polynomially many concurrent signing

		sk	pk	sig	Seq.OMUF	Con.OMUF
Tanuki 1 $\mu = 270$	CSIDH	32 B	64 B	8.47 KB	✓	$\log(\lambda)$
	LESS	32 B	13.6 KB	125 KB	✓	$\log(\lambda)$
Tanuki 2 $\kappa = \mu = 58$	CSIDH	1.78 KB	3.65 KB	1.86 KB	✓	$\log(\lambda)$
	LESS	1.78 KB	773 KB	26.8 KB	✓	$\log(\lambda)$
Tanuki 2 $\kappa = 16, \mu = 80$	CSIDH	480 B	960 B	2.54 KB	✓	$\log(\lambda)$
	LESS	480 B	203 KB	37.0 KB	✓	$\log(\lambda)$
Tanuki 3 $\kappa = 70, \mu = 70$	CSIDH	2.16 KB	4.4 KB	4.50 KB	✓	$\text{poly}(\lambda)$
	LESS	2.16 KB	936 KB	64.7 KB	✓	$\text{poly}(\lambda)$
Tanuki 3 $\kappa = 54, \mu = 108$	CSIDH	1.66 KB	3.31 KB	5.18 KB	✓	$\text{poly}(\lambda)$
	LESS	1.66 KB	719 KB	74.9 KB	✓	$\text{poly}(\lambda)$
CSI-Otter [70]		16 B	128 B	12* KB	N/A	$\log(\lambda)$
CSI-Otter- ζ_4 [70]		16 B	512 B	6* KB	N/A	$\log(\lambda)$
BlindOR		3.19 KB	34.3 KB	893 KB	N/A	$\log(\lambda)$
HKLN20 [65]		4.28 MB	444 KB	7.8 MB	N/A	7

Table 1. Comparison of key and signature sizes of different post-quantum sigma-protocol-based blind signatures for a 128-bit security level (i.e. $\lambda = 128$) and at most $Q = 2^{80}$ random oracle queries. For the performance of CSI-Otter [70], we scale their number of repetitions by 1.5 because the original number does not offer sufficient sequential security as mentioned in [52,71]. The last two columns indicate the sequential security and concurrent security respectively. The former notion means OMUF security by allowing only one execution at the same time. While the x-concurrent notion consider OMUF allow x signing session at the same time. In our LESS instantiations, the each secret key are highly compressible by using a 32-Byte seed and a PRNG.

sessions —addressing a major open challenge in the post-quantum blind signature landscape using the sigma-protocol approach. When instantiated with CSIDH-512, Framework **3** not only offers better signature size compared to CSI-Otter- ζ_4 , but also provides concurrent security.

Remark 3 (Other PQ Group Actions). Beyond CSIDH and LESS, several other candidates for post-quantum cryptographic group actions arise in the literature. For example, this includes MEDS [39] and ALTEQ [25]. It is worth mentioning that their underlying problems (the matrix code equivalence and the alternating trilinear form equivalence) are known to be polynomially equivalent.

In terms of representation sizes, LESS requires (472.5 B, 13.6 KB) to encode a group element and a set element, respectively. In comparison, plain MEDS and ALTEQ require (588, 4116) B and (676, 1144) B, respectively. Note that the parameters discussed here are those submitted by MEDS and ALTEQ in the first round of NIST standardization, but they are now considered insecure due to recent attacks [79,84]. Exploring alternative parameter sets that meet the security requirements is beyond the scope of this article. We leave a deeper investigation to future work. Another potential direction is the group action defined by the modulo lattice isomorphism problem used in the HAWK signature scheme. The main technical challenge lies in sampling group elements efficiently and correctly. While the group itself is infinite, the simulation for the non-modulo case (see Sec. 4.1 in [54]) is perfect. However, it is not clear if the corresponding

Metric	Tanuki 1 ($\mu = 270$)		Tanuki 2 ($\kappa = \mu = 58$)		Tanuki 2 ($\kappa = 16, \mu = 80$)		Tanuki 3 ($\kappa = 54, \mu = 108$)		Tanuki 3 ($\kappa = 70, \mu = 70$)	
	CSIDH (s)	LESS (ms)	CSIDH (s)	LESS (ms)	CSIDH (s)	LESS (ms)	CSIDH (s)	LESS (ms)	CSIDH (s)	LESS (ms)
KeyGen	0.027	0.26	1.55	13.1	0.41	2.81	1.42	9.49	1.83	12.7
Sign	21.9	157	4.70	41.5	6.39	47.2	13.3	94.8	11.3	81.0
Verify	6.92	54.2	1.48	14.0	2.06	16.1	4.10	32.8	3.51	28.0

Runtime in Mcycles

	CSIDH	LESS	CSIDH	LESS	CSIDH	LESS	CSIDH	LESS	CSIDH	LESS
KeyGen	74.0	0.59	4160	35.2	1104	8.13	3807	26.7	4909	35.1
Sign	58789	424	12622	112	17186	126	35865	252	30396	218
Verify	18604	145	3980	37.7	5526	43.6	11011	87.6	9433	75.6

Table 2. The runtime of our proof-of-concept implementation for CSIDH-512 (CSi-FiSh) (in seconds) and LESS (in milliseconds) group actions for blind signature schemes under the setting as Table 1 on an Intel Core Intel i7-12700H 2.30 GHz. The corresponding numbers of cycles are in the lower table.

sampler (Alg.6 of [53]) retains the desired properties. A full instantiation along these lines remains an open question for future exploration.

Remark 4 (Potential Improvements). While our main focus is on addressing theoretical bottlenecks in post-quantum blind signature design, we remark that the above size figures do not incorporate any scheme-specific optimizations.

First, as in the LESS implementation, each secret key can be compactly represented by a 32-byte seed. The light structure of monomial matrices gives an efficient decompression. This reduces the secret key size by $\approx 14\times$. Second, for isogeny-based schemes such as CSI-FiSh, it is possible to adopt the quadratic twist to reduce the signature size by roughly 37-50% [21]. Incorporating the technique would require minor modifications to the security proof and new selection of the parameters μ and κ as per theorems' statements. Lastly, in the LESS setting, one could use more compact representations as explored in [80,41], which significantly reduce the signature size by more than half. However, the representations seem not to preserve the group action structure considered in this paper. We leave the optimizations as potential future work.

Runtime. We implemented proof-of-concept versions of our schemes in C, and present benchmark results in Table 2. All benchmarks were conducted on an Intel Core i7-12700H 2.30 GHz CPU, using the compiler flags the flags `-O3 -mavx2 -mavx -march=native -mtune=native -ftree-vectorize`. For each LESS-based instantiation, we averaged over 1000 executions; for CSIDH-based instantiations, we averaged over 100 executions.

We use the reference implementation of CSiFiSh for our isogeny-based instantiations, which was performed on Window Subsystem Linux 2 running Ubuntu 24.04.2 LTS to enable the reference codes. For the code-based setting, we adapt the AVX2-optimized LESS implementation RREF functions from [8] into our specification and re-implement parts of the monomial computations ourselves. Overall, our results confirm that runtime is dominated by the number of group actions.

7 Conclusions

In this work, we have expanded the landscape of post-quantum blind signatures by exploring constructions from cryptographic group actions. We introduced four novel blind signature frameworks with varying assumptions, security, and efficiency trade-offs.

Our results demonstrate that group actions offer a flexible and powerful foundation for designing post-quantum secure blind signatures. In particular, we present the first instantiations from isogenies and codes that achieve concurrent security, addressing a major gap in the existing literature. These constructions provide potential alternatives to the currently dominant lattice-based construction using Fischlin’s paradigm. In addition, we outline several promising directions for further optimization and instantiation. While our randomized multi-key construction achieves polynomial concurrent security, it currently relies on the algebraic group action model and an interactive assumption. Removing these dependencies and achieving the same security level under non-interactive assumptions remains an important open problem. We hope the techniques and frameworks developed in this work lay a meaningful theoretical and practical foundation for future research.

Acknowledgments

Yi-Fu Lai is funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972. Lucjan Hanzlik and Eugenio Paracucchi were funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - 545066907. Gang Tang is supported by EPSRC through grant number EP/V011324/1.

References

1. National institute of standards and technology: Call for pqc standardization process (2017), <https://csrc.nist.gov/Projects/Post-Quantum-Cryptography>
2. National institute of standards and technology: Call for additional pq digital signature standardization process (2022), <https://csrc.nist.gov/Projects/pqc-dig-sig/standardization>
3. Agrawal, S., Kirshanova, E., Stehlé, D., Yadav, A.: Practical, round-optimal lattice-based blind signatures. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022: 29th Conference on Computer and Communications Security. pp. 39–53. ACM Press (Nov 2022). <https://doi.org/10.1145/3548606.3560650>
4. Alamati, N., De Feo, L., Montgomery, H., Patranabis, S.: Cryptographic group actions and applications. In: Moriai, S., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2020, Part II. Lecture Notes in Computer Science, vol. 12492, pp. 411–439. Springer, Cham (Dec 2020). https://doi.org/10.1007/978-3-030-64834-3_14

5. Alkeilani Alkadri, N., El Bansarkhani, R., Buchmann, J.: BLAZE: Practical lattice-based blind signatures for privacy-preserving applications. In: Bonneau, J., Heninger, N. (eds.) FC 2020: 24th International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 12059, pp. 484–502. Springer, Cham (Feb 2020). https://doi.org/10.1007/978-3-030-51280-4_26
6. Alkeilani Alkadri, N., El Bansarkhani, R., Buchmann, J.: On lattice-based interactive protocols: An approach with less or no aborts. In: Liu, J.K., Cui, H. (eds.) ACISP 20: 25th Australasian Conference on Information Security and Privacy. Lecture Notes in Computer Science, vol. 12248, pp. 41–61. Springer, Cham (Nov / Dec 2020). https://doi.org/10.1007/978-3-030-55304-3_3
7. Alkeilani Alkadri, N., Harasser, P., Janson, C.: BlindOR: an efficient lattice-based blind signature scheme from OR-proofs. In: Conti, M., Stevens, M., Krenn, S. (eds.) CANS 21: 20th International Conference on Cryptology and Network Security. Lecture Notes in Computer Science, vol. 13099, pp. 95–115. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92548-2_6
8. Baldi, M., Barenghi, A., Beckwith, L., BIASSE, J., Chou, T., Esser, A., Gaj, K., Karl, P., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.O., Santini, P., Wallace, R., Zweydinger, F.: LESS — Linear Equivalence Signature Scheme. Tech. rep., National Institute of Standards and Technology (2024), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>
9. Baldi, M., Barenghi, A., Beckwith, L., BIASSE, J., Esser, A., Gaj, K., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.O., Santini, P., Wallace, R.: LESS — Linear Equivalence Signature Scheme. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
10. Baldi, M., Barenghi, A., Beckwith, L., Chou, T., BIASSE, J.F., Esser, A., Gaj, K., Karl, P., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.J., Santini, P., Wallace, R., Zweydinger, F.: The less signature scheme (2023), <https://www.less-project.com/>
11. Barenghi, A., BIASSE, J.F., Ngo, T., Persichetti, E., Santini, P.: Advanced signature functionalities from the code equivalence problem. Cryptology ePrint Archive, Report 2022/710 (2022), <https://eprint.iacr.org/2022/710>
12. Battagliola, M., Borin, G., Di Crescenzo, G., Meneghetti, A., Persichetti, E.: Enhancing threshold group action signature schemes: Adaptive security and scalability improvements. Cryptology ePrint Archive, Report 2025/085 (2025), <https://eprint.iacr.org/2025/085>
13. Battagliola, M., Borin, G., Di Crescenzo, G., Meneghetti, A., Persichetti, E.: Enhancing threshold group action signature schemes: Adaptive security and scalability improvements. Cryptology ePrint Archive (2025)
14. Battagliola, M., Borin, G., Meneghetti, A., Persichetti, E.: Cutting the GRASS: Threshold GRoup action signature schemes. In: Oswald, E. (ed.) Topics in Cryptology – CT-RSA 2024. Lecture Notes in Computer Science, vol. 14643, pp. 460–489. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58868-6_18
15. Bauer, B., Fuchsbaauer, G., Plouviez, A.: The one-more discrete logarithm assumption in the generic group model. In: Tibouchi, M., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2021, Part IV. Lecture Notes in Computer Science, vol. 13093, pp. 587–617. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92068-5_20

16. Benčina, B., Budroni, A., Chi-Domínguez, J.J., Kulkarni, M.: Properties of lattice isomorphism as a cryptographic group action. In: Saarinen, M.J., Smith-Tone, D. (eds.) Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part I. pp. 170–201. Springer, Cham (Jun 2024). https://doi.org/10.1007/978-3-031-62743-9_6
17. Benhamouda, F., Lepoint, T., Loss, J., Orrù, M., Raykova, M.: On the (in)security of ROS. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021, Part I. Lecture Notes in Computer Science, vol. 12696, pp. 33–53. Springer, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-77870-5_2
18. Beullens, W.: Multivariate blind signatures revisited. IACR Cryptol. ePrint Arch. p. 720 (2024), <https://eprint.iacr.org/2024/720>
19. Beullens, W., Dobson, S., Katsumata, S., Lai, Y.F., Pintore, F.: Group signatures and more from isogenies and lattices: Generic, simple, and efficient. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022, Part II. Lecture Notes in Computer Science, vol. 13276, pp. 95–126. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_4
20. Beullens, W., Katsumata, S., Pintore, F.: Calamari and Falaf: Logarithmic (linkable) ring signatures from isogenies and lattices. In: Moriai, S., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2020, Part II. Lecture Notes in Computer Science, vol. 12492, pp. 464–492. Springer, Cham (Dec 2020). https://doi.org/10.1007/978-3-030-64834-3_16
21. Beullens, W., Kleinjung, T., Vercauteren, F.: CSI-FiSh: Efficient isogeny based signatures through class group computations. In: Galbraith, S.D., Moriai, S. (eds.) Advances in Cryptology – ASIACRYPT 2019, Part I. Lecture Notes in Computer Science, vol. 11921, pp. 227–247. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-34578-5_9
22. Beullens, W., Lyubashevsky, V., Nguyen, N.K., Seiler, G.: Lattice-based blind signatures: Short, efficient, and round-optimal. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023: 30th Conference on Computer and Communications Security. pp. 16–29. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3616613>
23. BIASSE, J.F., MICHELI, G., PERSICHETTI, E., SANTINI, P.: LESS is more: Code-based signatures without syndromes. In: Nitaj, A., Youssef, A.M. (eds.) AFRICACRYPT 20: 12th International Conference on Cryptology in Africa. Lecture Notes in Computer Science, vol. 12174, pp. 45–65. Springer, Cham (Jul 2020). https://doi.org/10.1007/978-3-030-51938-4_3
24. Bläser, M., Chen, Z., Duong, D.H., Joux, A., Nguyen, T.N., Plantard, T., Qiao, Y., Susilo, W., Tang, G.: On digital signatures based on group actions: QROM security and ring signatures. In: Saarinen, M.J., Smith-Tone, D. (eds.) Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part I. pp. 227–261. Springer, Cham (Jun 2024). https://doi.org/10.1007/978-3-031-62743-9_8
25. Bläser, M., Duong, D.H., Narayanan, A.K., Plantard, T., Qiao, Y., Sipasseuth, A., Tang, G.: ALTEQ. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
26. Bläser, M., Duong, D.H., Narayanan, A.K., Plantard, T., Qiao, Y., Sipasseuth, A., Tang, G.: The alteq signature scheme (2023), <https://pqcalteq.github.io/>
27. Bläser, M., Duong, D.H., Narayanan, A.K., Plantard, T., Qiao, Y., Sipasseuth, A., Tang, G.: The meds signature scheme (2023), <https://www.meds-pqc.org/>

28. Blazy, O., Gaborit, P., Schrek, J., Sendrier, N.: A code-based blind signature. In: 2017 IEEE International Symposium on Information Theory, ISIT 2017, Aachen, Germany, June 25-30, 2017. pp. 2718–2722. IEEE (2017). <https://doi.org/10.1109/ISIT.2017.8007023>, <https://doi.org/10.1109/ISIT.2017.8007023>
29. Boldyreva, A.: Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In: Desmedt, Y. (ed.) PKC 2003: 6th International Workshop on Theory and Practice in Public Key Cryptography. Lecture Notes in Computer Science, vol. 2567, pp. 31–46. Springer, Berlin, Heidelberg (Jan 2003). https://doi.org/10.1007/3-540-36288-6_3
30. Boneh, D., Guan, J., Zhandry, M.: A lower bound on the length of signatures based on group actions and generic isogenies. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023, Part V. Lecture Notes in Computer Science, vol. 14008, pp. 507–531. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_18
31. Bootle, J., Lyubashevsky, V., Seiler, G.: Algebraic techniques for short(er) exact lattice-based zero-knowledge proofs. In: Boldyreva, A., Micciancio, D. (eds.) Advances in Cryptology – CRYPTO 2019, Part I. Lecture Notes in Computer Science, vol. 11692, pp. 176–202. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26948-7_7
32. Brassard, G., Yung, M.: One-way group actions. In: Menezes, A.J., Vanstone, S.A. (eds.) Advances in Cryptology – CRYPTO’90. Lecture Notes in Computer Science, vol. 537, pp. 94–107. Springer, Berlin, Heidelberg (Aug 1991). https://doi.org/10.1007/3-540-38424-3_7
33. Budroni, A., Chi-Domínguez, J.J., D’Alconzo, G., Scala, A.J.D., Kulkarni, M.: Don’t use it twice! Solving relaxed linear equivalence problems. In: Advances in Cryptology – ASIACRYPT 2024, Part VIII. pp. 35–65. Lecture Notes in Computer Science, Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0944-4_2
34. Budroni, A., Chi-Domínguez, J.J., Franch, E.: Don’t use it twice: Reloaded! On the lattice isomorphism group action. Cryptology ePrint Archive, Report 2025/516 (2025), <https://eprint.iacr.org/2025/516>
35. Castryck, W., Lange, T., Martindale, C., Panny, L., Renes, J.: CSIDH: An efficient post-quantum commutative group action. In: Peyrin, T., Galbraith, S. (eds.) Advances in Cryptology – ASIACRYPT 2018, Part III. Lecture Notes in Computer Science, vol. 11274, pp. 395–427. Springer, Cham (Dec 2018). https://doi.org/10.1007/978-3-030-03332-3_15
36. Chairattana-Apirom, R., Hanzlik, L., Loss, J., Lysyanskaya, A., Wagner, B.: PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part III. Lecture Notes in Computer Science, vol. 13509, pp. 3–31. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15982-4_1
37. Chaum, D.: Blind signatures for untraceable payments. In: Chaum, D., Rivest, R.L., Sherman, A.T. (eds.) Advances in Cryptology – CRYPTO’82. pp. 199–203. Plenum Press, New York, USA (1982). https://doi.org/10.1007/978-1-4757-0602-4_18
38. Chen, M., Leroux, A., Panny, L.: SCALLOP-HD: Group action from 2-dimensional isogenies. In: Tang, Q., Teague, V. (eds.) PKC 2024: 27th International Conference on Theory and Practice of Public Key Cryptography, Part III. Lecture

- Notes in Computer Science, vol. 14603, pp. 190–216. Springer, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57725-3_7
39. Chou, T., Niederhagen, R., Persichetti, E., Ran, L., Randrianarisoa, T.H., Reijnders, K., Samardjiska, S., Trimoska, M.: MEDS — Matrix Equivalence Digital Signature. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
 40. Chou, T., Niederhagen, R., Persichetti, E., Randrianarisoa, T.H., Reijnders, K., Samardjiska, S., Trimoska, M.: Take your MEDS: Digital signatures from matrix code equivalence. In: El Mrabet, N., De Feo, L., Duquesne, S. (eds.) AFRICACRYPT 23: 14th International Conference on Cryptology in Africa. Lecture Notes in Computer Science, vol. 14064, pp. 28–52. Springer, Cham (Jul 2023). https://doi.org/10.1007/978-3-031-37679-5_2
 41. Chou, T., Persichetti, E., Santini, P.: On linear equivalence, canonical forms, and digital signatures. Cryptology ePrint Archive, Report 2023/1533 (2023), <https://eprint.iacr.org/2023/1533>
 42. Cong, K., Lai, Y.F., Levin, S.: Efficient isogeny proofs using generic techniques. In: Tibouchi, M., Wang, X. (eds.) ACNS 23: 21st International Conference on Applied Cryptography and Network Security, Part II. Lecture Notes in Computer Science, vol. 13906, pp. 248–275. Springer, Cham (Jun 2023). https://doi.org/10.1007/978-3-031-33491-7_10
 43. Couveignes, J.M.: Hard homogeneous spaces. IACR Cryptol. ePrint Arch. p. 291 (2006), <http://eprint.iacr.org/2006/291>
 44. Cozzo, D., Smart, N.P.: Sashimi: Cutting up CSI-FiSh secret keys to produce an actively secure distributed signing protocol. In: Ding, J., Tillich, J.P. (eds.) Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020. pp. 169–186. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-44223-1_10
 45. Cramer, R., Damgård, I., Schoenmakers, B.: Proofs of partial knowledge and simplified design of witness hiding protocols. In: Desmedt, Y. (ed.) Advances in Cryptology – CRYPTO’94. Lecture Notes in Computer Science, vol. 839, pp. 174–187. Springer, Berlin, Heidelberg (Aug 1994). https://doi.org/10.1007/3-540-48658-5_19
 46. D’Alconzo, G., Flamini, A., Gangemi, A.: Non-interactive commitment from non-transitive group actions. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology – ASIACRYPT 2023, Part VII. Lecture Notes in Computer Science, vol. 14444, pp. 222–252. Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8739-9_8
 47. Dartois, P., Eriksen, J.K., Fouotsa, T.B., Merdy, A.H.L., Invernizzi, R., Robert, D., Rueger, R., Vercauteren, F., Wesolowski, B.: PEGASIS: Practical effective class group action using 4-dimensional isogenies. Cryptology ePrint Archive, Paper 2025/401 (2025), <https://eprint.iacr.org/2025/401>
 48. Davidson, A., Goldberg, I., Sullivan, N., Tankersley, G., Valsorda, F.: Privacy pass: Bypassing internet challenges anonymously. Proceedings on Privacy Enhancing Technologies **2018**(3), 164–180 (Jul 2018). <https://doi.org/10.1515/popets-2018-0026>
 49. De Feo, L., Fouotsa, T.B., Kutas, P., Leroux, A., Merz, S.P., Panny, L., Wesolowski, B.: SCALLOP: Scaling the CSI-FiSh. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science, vol. 13940, pp. 345–375. Springer, Cham (May 2023). https://doi.org/10.1007/978-3-031-31368-4_13

50. De Feo, L., Galbraith, S.D.: SeaSign: Compact isogeny signatures from class group actions. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2019, Part III. Lecture Notes in Computer Science*, vol. 11478, pp. 759–789. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_26
51. De Feo, L., Meyer, M.: Threshold schemes from isogeny assumptions. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science*, vol. 12111, pp. 187–212. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45388-6_7
52. Do, K., Hanzlik, L., Paracucchi, E.: M&M’S: Mix and match attacks on Schnorr-type blind signatures with repetition. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology – EUROCRYPT 2024, Part VI. Lecture Notes in Computer Science*, vol. 14656, pp. 363–387. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58751-1_13
53. Ducas, L., Postlethwaite, E.W., Pulles, L.N., van Woerden, W.P.J.: Hawk: Module LIP makes lattice signatures fast, compact and simple. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022, Part IV. Lecture Notes in Computer Science*, vol. 13794, pp. 65–94. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22972-5_3
54. Ducas, L., van Woerden, W.P.J.: On the lattice isomorphism problem, quadratic forms, remarkable lattices, and cryptography. In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology – EUROCRYPT 2022, Part III. Lecture Notes in Computer Science*, vol. 13277, pp. 643–673. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07082-2_23
55. Duman, J., Hartmann, D., Kiltz, E., Kunzweiler, S., Lehmann, J., Riepel, D.: Generic models for group actions. In: Boldyreva, A., Kolesnikov, V. (eds.) *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science*, vol. 13940, pp. 406–435. Springer, Cham (May 2023). https://doi.org/10.1007/978-3-031-31368-4_15
56. Duong, D.H., Khuc, X.T., Qiao, Y., Susilo, W., Zhang, C.: Blind signatures from cryptographic group actions. *Cryptology ePrint Archive*, Paper 2025/397 (2025), <https://eprint.iacr.org/2025/397>
57. Espitau, T., Katsumata, S., Takemure, K.: Two-round threshold signature from algebraic one-more learning with errors. pp. 387–424. *Lecture Notes in Computer Science*, Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68394-7_-13
58. Feo, L.D., Meyer, M.: Threshold schemes from isogeny assumptions. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *Public-Key Cryptography - PKC 2020 - 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography*, Edinburgh, UK, May 4-7, 2020, *Proceedings*, Part II. *Lecture Notes in Computer Science*, vol. 12111, pp. 187–212. Springer (2020). https://doi.org/10.1007/978-3-030-45388-6_7, https://doi.org/10.1007/978-3-030-45388-6_7
59. Fischlin, M.: Round-optimal composable blind signatures in the common reference string model. In: Dwork, C. (ed.) *Advances in Cryptology – CRYPTO 2006. Lecture Notes in Computer Science*, vol. 4117, pp. 60–77. Springer, Berlin, Heidelberg (Aug 2006). https://doi.org/10.1007/11818175_4
60. Fuchsbaauer, G., Hanser, C., Kamath, C., Slamanig, D.: Practical round-optimal blind signatures in the standard model from weaker assumptions. In: Zikas, V.,

- De Prisco, R. (eds.) SCN 16: 10th International Conference on Security in Communication Networks. Lecture Notes in Computer Science, vol. 9841, pp. 391–408. Springer, Cham (Aug / Sep 2016). https://doi.org/10.1007/978-3-319-44618-9_21
61. Fuchsbauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: Shacham, H., Boldyreva, A. (eds.) *Advances in Cryptology – CRYPTO 2018, Part II. Lecture Notes in Computer Science*, vol. 10992, pp. 33–62. Springer, Cham (Aug 2018). https://doi.org/10.1007/978-3-319-96881-0_2
 62. Goldreich, O., Micali, S., Wigderson, A.: Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM* **38**(3), 691–729 (Jul 1991). <https://doi.org/10.1145/116825.116852>
 63. Hanzlik, L., Loss, J., Wagner, B.: Rai-choo! Evolving blind signatures to the next level. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023, Part V. Lecture Notes in Computer Science*, vol. 14008, pp. 753–783. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_26
 64. Hauck, E., Kiltz, E., Loss, J.: A modular treatment of blind signatures from identification schemes. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2019, Part III. Lecture Notes in Computer Science*, vol. 11478, pp. 345–375. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_12
 65. Hauck, E., Kiltz, E., Loss, J., Nguyen, N.K.: Lattice-based blind signatures, revisited. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology – CRYPTO 2020, Part II. Lecture Notes in Computer Science*, vol. 12171, pp. 500–529. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_18
 66. Heilman, E., Alshenibr, L., Baldimtsi, F., Scafuro, A., Goldberg, S.: TumbleBit: An untrusted bitcoin-compatible anonymous payment hub. In: *ISOC Network and Distributed System Security Symposium – NDSS 2017. The Internet Society* (Feb / Mar 2017). <https://doi.org/10.14722/ndss.2017.23086>
 67. Ji, Z., Qiao, Y., Song, F., Yun, A.: General linear group action on tensors: A candidate for post-quantum cryptography. In: Hofheinz, D., Rosen, A. (eds.) *TCC 2019: 17th Theory of Cryptography Conference, Part I. Lecture Notes in Computer Science*, vol. 11891, pp. 251–281. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-36030-6_11
 68. Kastner, J., Loss, J., Xu, J.: The Abe-Okamoto partially blind signature scheme revisited. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022, Part IV. Lecture Notes in Computer Science*, vol. 13794, pp. 279–309. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22972-5_10
 69. Kastner, J., Loss, J., Xu, J.: On pairing-free blind signature schemes in the algebraic group model. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science*, vol. 13178, pp. 468–497. Springer, Cham (Mar 2022). https://doi.org/10.1007/978-3-030-97131-1_16
 70. Katsumata, S., Lai, Y.F., LeGrow, J.T., Qin, L.: CSI-Otter: Isogeny-based (partially) blind signatures from the class group action with a twist. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part III. Lecture Notes in Computer Science*, vol. 14083, pp. 729–761. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38548-3_24
 71. Katsumata, S., Lai, Y.F., Reichle, M.: Breaking parallel ROS: Implication for isogeny and lattice-based blind signatures. In: Tang, Q., Teague, V. (eds.) *PKC 2024: 27th International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science*, vol. 14601, pp. 319–351. Springer, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57718-5_11

72. Knuth, D.: The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1. Pearson Education India (2011), https://books.google.de/books?id=7k6_1UUpnLsC
73. Kuchta, V., LeGrow, J.T., Persichetti, E.: Post-quantum blind signatures from matrix code equivalence. Cryptology ePrint Archive, Paper 2025/274 (2025), <https://eprint.iacr.org/2025/274>
74. Lai, Y.F.: Cappybara and tsubaki: Verifiable random functions from group actions and isogenies. IACR Communications in Cryptology (CiC) 1(3), 1 (2024). <https://doi.org/10.62056/avr-11zn4>
75. Lyubashevsky, V.: Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In: Matsui, M. (ed.) Advances in Cryptology – ASIACRYPT 2009. Lecture Notes in Computer Science, vol. 5912, pp. 598–616. Springer, Berlin, Heidelberg (Dec 2009). https://doi.org/10.1007/978-3-642-10366-7_35
76. Lyubashevsky, V., Nguyen, N.K., Plançon, M.: Efficient lattice-based blind signatures via gaussian one-time signatures. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol. 13178, pp. 498–527. Springer, Cham (Mar 2022). https://doi.org/10.1007/978-3-030-97131-1_17
77. Lyubashevsky, V., Nguyen, N.K., Plançon, M.: Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 71–101. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15979-4_3
78. Meers, J., Riepel, D.: CCA secure updatable encryption from non-mappable group actions. In: Saarinen, M.J., Smith-Tone, D. (eds.) Post-Quantum Cryptography – 15th International Workshop, PQCrypto 2024, Part I. pp. 137–169. Springer, Cham (Jun 2024). https://doi.org/10.1007/978-3-031-62743-9_5
79. Narayanan, A.K., Qiao, Y., Tang, G.: Algorithms for matrix code and alternating trilinear form equivalences via new isomorphism invariants. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, Part III. Lecture Notes in Computer Science, vol. 14653, pp. 160–187. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58734-4_6
80. Persichetti, E., Santini, P.: A new formulation of the linear equivalence problem and shorter LESS signatures. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology – ASIACRYPT 2023, Part VII. Lecture Notes in Computer Science, vol. 14444, pp. 351–378. Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8739-9_12
81. Petzoldt, A., Szeplieniec, A., Mohamed, M.S.E.: A practical multivariate blind signature scheme. In: Kiayias, A. (ed.) FC 2017: 21st International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 10322, pp. 437–454. Springer, Cham (Apr 2017). https://doi.org/10.1007/978-3-319-70972-7_25
82. del Pino, R., Katsumata, S.: A new framework for more efficient round-optimal lattice-based (partially) blind signature via trapdoor sampling. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 306–336. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15979-4_11

83. Pointcheval, D., Stern, J.: Security arguments for digital signatures and blind signatures. *Journal of Cryptology* **13**(3), 361–396 (Jun 2000). <https://doi.org/10.1007/s001450010003>
84. Ran, L., Samardjiska, S.: Rare structures in tensor graphs - bermuda triangles for cryptosystems based on the tensor isomorphism problem. In: *Advances in Cryptology – ASIACRYPT 2024, Part VIII*. pp. 66–96. *Lecture Notes in Computer Science*, Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0944-4_3
85. Rückert, M.: Lattice-based blind signatures. In: Abe, M. (ed.) *Advances in Cryptology – ASIACRYPT 2010*. *Lecture Notes in Computer Science*, vol. 6477, pp. 413–430. Springer, Berlin, Heidelberg (Dec 2010). https://doi.org/10.1007/978-3-642-17373-8_24
86. Tang, G., Duong, D.H., Joux, A., Plantard, T., Qiao, Y., Susilo, W.: Practical post-quantum signature schemes from isomorphism problems of trilinear forms. In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology – EUROCRYPT 2022, Part III*. *Lecture Notes in Computer Science*, vol. 13277, pp. 582–612. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07082-2_21
87. W.Bos, J., Bronchain, O., Ducas, L., Fehr, S., Huang, Y.H., Pornin, T., W.Postlethwaite, E., Prest, T., N.pulles, L., van Woerden, W.: The hawk signature scheme (2023), <https://hawk-sign.info/>

A Related Work

Group actions in (post-quantum) cryptography. Cryptographic group actions, though less explored than traditional group-theoretic assumptions, have garnered attention in recent years due to their potential in constructing post-quantum cryptographic primitives. Brassard and Yung [32] introduced the notion of *one-way* group action in 1990, marking the beginning of this line of research. Since then, group actions have been applied to a variety of cryptographic primitives such as commitment schemes [46,67], (ring) signatures [20,24], threshold signatures [14,44,58,13], etc. When the groups are abelian, the group action was further developed by Couveignes [43]. Simultaneously, the inclusion of additional properties expands the potential for constructing a broader range of cryptographic primitives, such as key exchange [35,43], blind signature [70], dual-mode public-key encryption [4], etc.

The recent PQC standardization [1] led by National Institute of Standards and Technology (NIST), has reinvigorated interest within the cryptographic community in the design of efficient post-quantum cryptographic schemes. This also promotes community’s attention to group action in post-quantum cryptography, especially the diversity of assumptions explicitly pointed out in the latest call for standardization [2]. Driven by this impetus, a variety of group actions have emerged within the realm of post-quantum cryptography, incorporating a wide of algebraic objects, including lattices [54,16], linear codes [23,40], isogenies [35], and trilinear forms [86]. Notably, in the latest of the NIST standardization, three schemes from the first round candidates are based on group actions: MEDS [27], LESS [10], and ALTEQ [26]. The three schemes adopt the same foundational paradigm, that is, the Goldreich–Micali–Widgerson (GMW) zero-knowledge protocol and the Fiat–Shamir transformation. While HAWK [87] is constructed using the Hash-and-Sign paradigm, it also offers the cryptographic group action properties. It is worth mentioning that two of these schemes, i.e., LESS and HAWK, have advanced to the second round of candidates in the latest NIST standardization.

Post-quantum blind signatures. Lattice-based cryptography has emerged as a promising approach for constructing post-quantum blind signature schemes. The first such scheme was introduced by Rükert [85], drawing inspiration from classical Schnorr and Okamoto-Schnorr [83] blind signatures. Following Rükert’s design blueprint, a series of optimizations lead to further works [5,7]. However, the security proofs of this series of works were recently found to have errors [65]. The construction by Hauck et al. [65] aimed to fix the errors, thus they subsequently introduced the first provably secure construction in this paradigm, albeit with impractically large signatures size $\approx 7.73\text{MB}$, and for security against adversaries limited to getting 7 signatures.

Recently, Lyubashevsky et al. [76] employed one-time signatures and OR proofs to significantly reduce signature size to 150 KB, albeit with a bounded number of signatures per public key. Concurrently, del Pino and Katsumata [82] introduced a blind signature scheme based on standard lattice assumptions,

achieving a signature size $\approx 100\text{KB}$ and a total communication cost under one megabyte. Agrawal et al. [3] improved upon this by using the one-more SIS assumption, yielding a more efficient scheme with a signature size $\approx 45\text{KB}$ and lower communication complexity. Further optimizations were achieved by Beullens et al. [22], who introduced a 2-round lattice-based blind signature scheme with a signature size of just 22 KB. This construction, which relies on an NIZK for proving relations of concrete hash functions, is currently the most compact among lattice-based blind signatures. This protocol is round-optimal and has a transcript size that can be as small as 60 KB. Additionally, Beullens et al. proposed a keyed-verification variant where the signature size is minimized to 48 bytes. But compared to basic blind signatures, this variant requires computing 100x more hash functions, making it impractical at this stage.

Beyond lattices, blind signatures in the post-quantum literature have been sparse. An early attempt by Blazy et al. [28] introduced a code-based blind signature scheme following Fischlin’s generic construction, employing a zero-knowledge proof framework adapted for concatenated matrices. While viable, the signature size is relatively large, particularly for Hamming metric-based constructions. Another notable work by Petzoldt et al. [81] proposed a multivariate-based blind signature scheme derived from the Rainbow signature scheme. Specifically, this construction is based on a multivariate trapdoor and a zero-knowledge proof system. Very recently, Beullens [18] found this construction is vulnerable to attack due to the fact that multivariate quadratic maps are not binding. Recently, [70] proposed the first blind signature schemes from isogenies with a signature size of 4 KB, but it was shown to not provide concurrent security for polynomially many signatures.

B Omitted Preliminaries

Lemma 3 (Stirling Approximation). *For $n \in \mathbb{N}$, we have $n! \approx \sqrt{2\pi e} \left(\frac{n}{e}\right)^n e^{\frac{1}{12n}}$.*

Lemma 4. *For $n \in \mathbb{N}$, we have $\binom{n}{n/2} \approx 2^n / \sqrt{\frac{\pi n}{2}}$.*

Lemma 5. *For $\alpha, \kappa \in \mathbb{N}$ we have*

$$\binom{\alpha \cdot \kappa}{\alpha, \dots, \alpha} = \frac{(\alpha \cdot \kappa)!}{\alpha!^\kappa} \approx \frac{\kappa^{\kappa\alpha+1/2}}{\sqrt{(2\pi\alpha)^{\kappa-1}}}.$$

C Concurrent vs. Sequential Security

We will now discuss the differences between concurrent and sequential security in the context of blind signatures. We begin with a simple observation that concurrent security implies sequential security, i.e., any scheme that is secure against adversaries that are allowed to launch concurrent attacks is also safe against sequential attacks. More formally, the following lemma holds.

Lemma 6. *Let BS be a blind signature scheme and assume two p.p.t. adversaries $\mathcal{A}_c$ and $\mathcal{A}_s$ respectively from the cOMUF and sOMUF experiments. If $\text{Adv}_{\mathcal{A}_c, \ell}^{\text{cOMUF}}(\lambda) \leq \text{negl}(\lambda)$ then we have $\text{Adv}_{\mathcal{A}_s, \ell}^{\text{sOMUF}}(\lambda) \leq \text{negl}(\lambda)$.*

Proof. Assume by contradiction that the advantage of adversary $\mathcal{A}_s$ is not negligible in winning the sOMUF experiment. We will now use this adversary to construct adversary $\mathcal{A}_c$ for which the $\text{Adv}_{\mathcal{A}_c, \ell}^{\text{cOMUF}}(\lambda)$ is also non-negligible. The transformation is straightforward since $\mathcal{A}_s$ is also an adversary in the cOMUF experiment since the only difference between experiments is that condition on the Sign_1 oracle.

It is worth noting that the opposite claim is not valid. We will now discuss the main result of this subsection. The authors of [52] noted that while the CSI-Otter scheme is not practical in the concurrent setting because of the attack they describe, they propose to use CSI-Otter in the sequential setting. The intuition here is that since concurrent security is stronger, even if a scheme is secure for small ℓ , it should be sequentially secure for larger ℓ . Unfortunately, we will show that this is untrue, and sequential security needs to be proven separately for larger ℓ values. In particular, ℓ concurrent unforgeability does not even imply $\ell + 1$ sequential unforgeability.

Lemma 7. *Given an integer ℓ , there exists a blind signature scheme, where for any p.p.t. adversary $\mathcal{A}$, we have $\text{Adv}_{\mathcal{A}, \ell}^{\text{cOMUF}}(\lambda) \leq \text{negl}(\lambda)$, but $\text{Adv}_{\mathcal{A}, \ell+1}^{\text{sOMUF}}(\lambda) = 1$.*

Proof (Sketch). We omit formal proofs and focus on intuition. Our approach builds on Fischlin’s framework [59], which generically constructs blind signatures using extractable non-interactive zero-knowledge (NIZK) proofs, a commitment scheme, and a standard signature scheme. In this framework, the user receives a signature on a commitment to its message. To ensure blindness, the final signature is a non-interactive zero-knowledge proof of knowledge of a signature under a commitment that opens to the message signed.

For our proof, we slightly modify this setup by introducing a *trapdoor witness*, which enables us to generate valid proofs for any message an adversary chooses, leading to unlimited forgeries. This trapdoor witness consists of knowledge of $\ell + 1$ signatures on unique messages. Notably, this witness is useless to an adversary constrained by the ℓ -one-more-unforgeability notion, as they can only learn up to ℓ signatures. Thus, the original security arguments from [59] still hold, proving ℓ -one-more unforgeability in the concurrent setting.

However, this does not imply the scheme is $\ell + 1$ one-more unforgeable in the sequential setting. In particular, the attacker can use the $\ell + 1$ queries provided by the challenger to generate the trapdoor witness and later create forged proof and signatures at will, easily breaking the $\ell + 1$ one-more unforgeability. This completes the proof.

D Fourth Framework: OR-Proof Construction

In this subsection, we introduce a simple modification to Framework 1 that bases its security on the GAIP assumption. The core idea is to use the OR proof technique.

At first glance, one might expect that this approach could be easily applied to both Frameworks 1 and 2. However, we remark that this is not the case. The primary challenge arises from the use of the parallel OR-proof, which requires performing “group operations” between the challenge strings. In our case, the challenge space is unbalanced and is naturally endowed with group operations.

Although it is possible to construct an efficient mapping and inverse mapping between the challenge space and $\mathbb{Z}_n$ —where n is the size of the challenge space—by leveraging its combinatorial structure (e.g., Sec 7. [72]), this does not straightforwardly provide a corresponding de-masking mechanism for User₁. The difficulty lies in the fact that the user must permute the hash output according to the predetermined permutations in advance while the user has two masking permutations using the OR-proof.

However, we observe that under a specific circumstance, it is possible to define a group operation over the challenge space that remains compatible with the permutation operation. This observation allows for a structured approach in applying the OR-proof technique to Framework 1.

The idea of the following scheme (Framework 4) is to exploit the special case of Framework 1 for $\mu = 2$, where the challenge space is $\{(0, 1), (1, 0)\}$. In this case, the permutation over $\{0, 1\}$ can be expressed as XOR with a bit $\rho \in \{0, 1\}$. Concretely, the operation $(0 \oplus \rho, 1 \oplus \rho)$ represents a transposition if $\rho = 1$, and the identity map otherwise. Meanwhile, we can encode the challenge space $\{(0, 1), (1, 0)\}$ into $\{0, 1\}$, where the group operation (i.e., $\oplus$) on the challenge space commutes with the permutation operation (i.e., $\oplus$). This property allows us to maintain compatibility between the challenge structure and the permutation mechanism. Then, we apply a standard parallel repetition technique, ensuring that the scheme achieves sufficient security strength. Finally, we show the correctness proof and establish the one-more unforgeability (OMUF) property. The proof of blindness is omitted due to its similarity to previous proofs.

To simplify the description, we introduce a standard graph-isomorphism-type zero-knowledge simulator *Sim*. The algorithm *Sim* on input a pair of set elements (pk_0, pk_1) and a string $c' \in \{0, 1\}^\mu$ returns $((y_{i,t})_{i,t \in [\mu] \times \{0,1\}}, (s'_{i,t})_{i,t \in [\mu] \times \{0,1\}})$ such that $s'_{i,t} \star pk_{t \oplus c'_{i,t}} = y_{i,t}$ for all $i, t \in [\mu] \times \{0, 1\}$. The simulated transcripts are perfectly indistinguishable from the real transcripts generated during the interaction between the prover and the verifier.

Notations. We introduce a notation for the subsection. For a string $\rho = (\rho_0, \dots, \rho_n)$ over $\{0, 1\}^n$ and a array $(z_{1,0}, z_{0,1}, \dots, z_{n,0}, z_{n,1})$ of length n , we denote $\rho(z) := (z_{1,0 \oplus \rho_1}, z_{0,1 \oplus \rho_1}, \dots, z_{n,0 \oplus \rho_n}, z_{n,1 \oplus \rho_n})$.

Theorem 11. *The blind signature construction presented in Framework 4 has correctness.*

Let $(x, X, G, \star)$ be a cryptographic group action. Let $\mu = \mu(\lambda)$ be polynomial functions in the security parameter λ . Let the hash function H map from $\{0, 1\}^*$ onto $\text{ChSet} = \{0, 1\}^\mu$. Let H be modeled as a random oracle.

KeyGen(λ): $b \leftarrow \{0, 1\}$ set $g_{0,0} = g_{1,0} = 1$ and sample group elements $g_{0,1}, g_{1,1} \leftarrow \mathbb{G}^2$, output $\text{sk} = (b, g_{b,1})$ and $\text{pk} = (\text{pk}_0, \text{pk}_1) = ((g_{0,0}, g_{0,1}) \star x, (g_{1,0}, g_{1,1}) \star x)$.

Signer₁(sk): sample group elements $h_b = (h_{b,1,0}, h_{b,1,1}, \dots, h_{b,\mu,0}, h_{b,\mu,1}) \leftarrow \mathbb{G}^{2\mu}$. Compute $Y_b = h_b \star x \in X^{2\mu}$. Sample $c'_b = (c'_{b,1}, \dots, c'_{b,\mu}) \leftarrow \text{ChSet}$ and run $(Y_b, s'_b) \leftarrow \text{Sim}(\text{pk}_b, c'_b)$. Send $Y = (Y_0, Y_1) = ((y_{0,1,0}, y_{0,1,1}, \dots, y_{0,\mu,0}, y_{0,\mu,1}), (y_{1,1,0}, y_{1,1,1}, \dots, y_{1,\mu,0}, y_{1,\mu,1}))$ to User.

User₁(pk, Y, m): sample group elements $(r_0, r_1) = ((r_{0,1,0}, \dots, r_{0,\mu,1}), (r_{1,1,0}, \dots, r_{1,\mu,1})) \leftarrow G^{4\mu}$ and random strings ρ_0, ρ_1 over ChSet . Compute $(z_0, z_1) = (r_0 \star Y_0, r_1 \star Y_1)$. Compute the challenge $C = H((\rho_0(z_0) \parallel \rho_1(z_1) \parallel m))$ and return $C' = C \oplus \rho_0 \oplus \rho_1$

Signer₂(sk, C'): set $C' \oplus c'_b = c'_b$. Parse $c'_b = (c'_{b,1}, \dots, c'_{b,\mu})$ and for each $i \in [\mu]$ set

$$s'_{b,i,0} = h_{b,i,0} \cdot g_{0 \oplus c'_{b,i}}^{-1}; \quad s'_{b,i,1} = h_{b,i,1} \cdot g_{1 \oplus c'_{b,i}}^{-1}$$

Send $S' = ((c'_0, c'_1), (s'_{0,1,0}, \dots, s'_{0,\mu,1}), (s'_{1,1,0}, \dots, s'_{1,\mu,1}))$ to the User.

User₂(pk, S', m): parse $S' = ((c'_0, c'_1), (s'_0, s'_1))$. Return $\perp$ if $s'_{b',i,t} \star \text{pk}_{b,t \oplus c'_{b,t}} = y_{b',i,t}$ does not hold for any $(b', i, t) \in \{0, 1\} \times [\mu] \times \{0, 1\}$. Otherwise, compute $(s_0, s_1) = (r_0 \cdot s'_0, r_1 \cdot s'_1)$.

Return the signature $\text{sig} = ((\rho_0(z_0), \rho_1(z_1)), (\rho_0 \oplus c'_0, \rho_1 \oplus c'_1), (\rho_0(s_0), \rho_1(s_1)))$

Verify(pk, m, sig): Parse sig as $((\tilde{z}_0, \tilde{z}_1), (c_0, c_1), (\tilde{s}_0, \tilde{s}_1))$ where

$$((\tilde{z}_0, \tilde{z}_1), (\tilde{s}_0, \tilde{s}_1)) = ((\tilde{z}_{0,1}, \dots, \tilde{z}_{0,\mu}), (\tilde{z}_{1,1}, \dots, \tilde{z}_{1,\mu}), (\tilde{s}_{0,1}, \dots, \tilde{s}_{0,\mu}), (\tilde{s}_{1,1}, \dots, \tilde{s}_{1,\mu}))$$

and compute the challenge $c_0 \oplus c_1 = H(\tilde{z}_0 \parallel \tilde{z}_1 \parallel m)$. Output 1 if for all $i \in [\mu]$ the following holds:

$$\begin{aligned} \tilde{s}_{0,i,0} \star \text{pk}_{0,0 \oplus c_{0,i}} &= \tilde{z}_{0,i,0} \wedge \tilde{s}_{0,i,1} \star \text{pk}_{0,1 \oplus c_{0,i}} = \tilde{z}_{0,i,1} \\ \tilde{s}_{1,i,0} \star \text{pk}_{1,0 \oplus c_{1,i}} &= \tilde{z}_{1,i,0} \wedge \tilde{s}_{1,i,1} \star \text{pk}_{1,1 \oplus c_{1,i}} = \tilde{z}_{1,i,1}. \end{aligned}$$

Otherwise, output 0.

Framework 4: Binary OR-rized Blind Signature using DLog

Proof. We use the same notations as described in Framework 4. The correctness proof follows by ensuring that the verification conditions hold whenever both parties follow the protocol.

To verify correctness, we first show for each $i \in [\mu]$:

$$s_{0,i,0 \oplus \rho_{0,i}} \star \text{pk}_{0,0 \oplus c_{0,i}} = z_{0,i,0 \oplus \rho_{0,i}}, s_{0,i,1 \oplus \rho_{0,i}} \star \text{pk}_{0,1 \oplus c_{0,i}} = z_{0,i,1 \oplus \rho_{0,i}}, \quad (8)$$

$$s_{1,i,0 \oplus \rho_{1,i}} \star \text{pk}_{1,0 \oplus c_{1,i}} = z_{1,i,0 \oplus \rho_{1,i}}, s_{1,i,1 \oplus \rho_{1,i}} \star \text{pk}_{1,1 \oplus c_{1,i}} = z_{1,i,1 \oplus \rho_{1,i}}. \quad (9)$$

Expanding the expressions, we observe that $s'_{b',i,t} \star \text{pk}_{b,t \oplus c'_{b,t}} = y_{b',i,t}$ for all $(b', i, t) \in \{0, 1\} \times [\mu] \times \{0, 1\}$ where the equality holds for $b' = \bar{b}$ due to the property of the simulator. Hence, we have $(r_{b',i,t} \cdot s'_{b',i,t}) \star \text{pk}_{b,t \oplus c'_{b,t}} = z_{b,i,t}$ so that $\tilde{z}_{b',i,t} = s_{0,i,t \oplus \rho_{0,i}} \star \text{pk}_{0,t \oplus c_{b,i}}$ due to $c'_{b'} = c_{b'} \oplus \rho_{b'}$.

Besides, since $H(\tilde{z}_0 \| \tilde{z}_1 \| m) \oplus \rho_0 \oplus \rho_1 = c'_0 \oplus c'_1$, $c_0 \oplus \rho_0 = c'_0$, and $c_1 \oplus \rho_1 = c'_1$, we have $H(\tilde{z}_0 \| \tilde{z}_1 \| m) = c_0 \oplus c_1$. Therefore, the scheme satisfies correctness.

Theorem 12. *Given a p.p.t. adversary $\mathcal{A}$ against the blind signature construction Framework 4 over the random oracle model and the algebraic group action model using Q random oracle queries, there exists an GAIP adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\mathcal{A},\ell}^{\text{sOMUF}}(\lambda) \leq 2 \cdot \text{Adv}_{\mathcal{B}}^{\text{GAIP}} + \frac{Q + \ell + 1}{2^\mu} + \frac{\binom{Q}{2}}{2^\mu}.$$

where $\mathcal{B}$ runs polynomially in λ, μ, ℓ and the approximation is based on the Stirling approximation.

Proof. Let $\mathcal{A}$ be a p.p.t. algebraic adversary against sequential one-more unforgeability. We construct a GAIP adversary $\mathcal{B}$ with access to $\mathcal{A}$ proceeding as follows.

1. $\mathcal{B}$ invoke $\mathcal{A}$ with the public parameter $(x, X, G, \star)$ obtained from the GAIP game. $\mathcal{B}$ also simulates the random oracle via the standard lazy sampling method. That is, on input a set vector $\overrightarrow{\text{com}}$ and a message m , $\mathcal{B}$ records the representation for each set element within $\overrightarrow{\text{com}}$, reply with a string uniformly randomly from the codomain, and maintain the query history.
2. After obtaining the GAIP challenge x' , $\mathcal{B}$ proceeds the same as **KeyGen** and obtain $\text{sk} = (b, g_{b,1})$ and $(\text{pk}_0 = (x, x_1), \text{pk}_1 = (x, x'))$ except that it replaces $\text{pk}_{\bar{b}} \leftarrow (x, x')$. Then, $\mathcal{B}$ returns the public key $(\text{pk}_0, \text{pk}_1)$ to the adversary. We denote the strings returned by **Signer**₂ as $(\mathbf{c}'_{j,0}, \mathbf{c}'_{j,1})$.
3. For $j \in [\ell]$, in response to $\mathcal{A}$'s signing queries, $\mathcal{B}$ runs **Signer** on input sk .
4. $\mathcal{B}$ receives the $\ell + 1$ forgeries $\{(m_j, (\overrightarrow{\text{com}}_{j,t}, \overrightarrow{\text{ch}}_{j,t}, \overrightarrow{\text{resp}}_{j,t})_{t \in \{0,1\}})\}_{j \in [\ell+1]}$, where for $j \in [\ell + 1]$ we have $(\overrightarrow{\text{com}}_{j,t}, \overrightarrow{\text{ch}}_{j,t}, \overrightarrow{\text{resp}}_{j,t})$ satisfies one of the verification Eqs. (8) and (9) up to $t \in \{0, 1\}$.
5. To find $g \in G$ such that $g \star x = x'$, $\mathcal{B}$ checks when the following equation does not hold

$$\mathbf{A}_{0,j} \cdot (0, 1, \mathbf{c}'_0)^\top \oplus \mathbf{A}_{1,j} \cdot (0, 1, \mathbf{c}'_1)^\top =_{\mathbf{B}} H(\overrightarrow{\text{com}}_{j,0} \| \overrightarrow{\text{com}}_{j,1} \| m_j) \quad (10)$$

where $\mathbf{A}_{0,j}, \mathbf{A}_{1,j}$ are constructed in the same way as Eq. (4) using algebraic adversary's auxiliary output and $\mathbf{c}'_0 = (\mathbf{c}'_{1,0}, \dots, \mathbf{c}'_{\ell,0})$, $\mathbf{c}'_1 = (\mathbf{c}'_{1,1}, \dots, \mathbf{c}'_{\ell,1})$.

By using the same approach as in the proof of Theorem 5, we can extract an element $g \in G$ such that $g \star x = x_1$ or x' . We remark that the scheme is *witness indistinguishable* [45, Thm.8], which means if $\mathcal{B}$ execute the scheme using the witness of x' rather than of x_1 , the executions follows the same distribution. Hence, the secret bit b is information-theoretically hidden from the adversary and is randomly drawn from $\{0, 1\}$, and the success rate of the extraction is $1/2$.

It suffices to show Eq. (10) holds with a negligible chance over the random oracle model. To this end, we define the following three events.

- We define E_1 as the event where $\exists j \in [\mu]$ such that the query $(\overrightarrow{\text{com}}_{j,0} \| \overrightarrow{\text{com}}_{j,1} \| m_j)$ has never been made to H by $\mathcal{A}$.
- We define E_2 as the event where for some $j \in [\mu]$, the query $(\overrightarrow{\text{com}}_{j,0} \| \overrightarrow{\text{com}}_{j,1} \| m_j)$ is first made during the gap when no sessions are open.
- We define E_3 as the event where for some $j \in [\mu]$, the query $(\overrightarrow{\text{com}}_{j,0} \| \overrightarrow{\text{com}}_{j,1} \| m_j)$ is first made after the beginning of a session and before its closure.

Success conditioned on E_1 . Assuming there does not exist an element $g \in G$ and its corresponding set element $z \in X$ in the forgeries such that both $g \star x = z$ and $g \star z = x'$ hold (as this would provide a GAIP solution between x and x'). Hence at most one equality out of two in the verification holds with the given forgeries. Modeling H as a random oracle, for each $j \in [\mu]$ the event E_1 happens with a probability of $\frac{1}{2^\mu}$. By the union bound, the probability is bounded above by $\frac{\ell+1}{2^\mu}$.

Success conditioned on $E_2 - E_1$. If the event occurs right before the beginning of the J -th session for some $J \in [\ell + 1]$, the corresponding $\mathbf{c}'_{j,0}, \mathbf{c}'_{j,1}$ values have been determined for every $j \in [J - 1]$ and $i \in [\mu]$. The probability that such a query results in a forgery such that Eq. (10) holds is less than $\frac{1}{2^\mu}$. This result holds the same if the query is made after the last session. By the union bound, the probability is bounded above by $\frac{Q}{2^\mu}$.

Success conditioned on $E_3 - E_1 - E_2$. Since there are ℓ sessions with $\ell + 1$ hash queries in the forgeries, there are at least two such queries made within the same session. For any two queries made within the J -th session, the corresponding $\mathbf{c}'_{j,0}, \mathbf{c}'_{j,1}$ values in Eq. (10) have been determined for every $j \in [J - 1]$ and $i \in [\mu]$. Moreover, the value of $\mathbf{c}'_{J,\bar{b}}$ has been drawn uniformly at random during the execution of Signer_1 . It follows that the corresponding two matrices of the two queries result in at most 2^μ vectors over $\{0, 1\}^{2^\mu}$, came from the undetermined $\mathbf{c}'_{J,\bar{b}}$. Hence, Eq. (4) holds with probability less than $\frac{2^\mu}{2^{2^\mu}}$. By the union bound, the probability is bounded above by $\frac{Q}{2^\mu}$. Hence, we have

$$\text{Adv}_{\mathcal{A},\ell}^{\text{sOMUF}}(\lambda) \leq 2 \cdot \text{Adv}_{\mathcal{B}}^{\text{GAIP}} + \frac{Q + \ell + 1}{2^\mu} + \frac{\binom{Q}{2}}{2^\mu}.$$

E Proofs for Framework 2

E.1 Proof of correctness

Proof. We show that Framework 2 is correct. Let $\mathbf{pk} = (g_1 \star x, \dots, g_\kappa \star x)$ be the public key, and let $m, \mathbf{sig}$ be the signature output by User_2 following the protocol. We have

$$\mathbf{sig} = (z_{\pi(1)}, \dots, z_{\pi(\mu)}), (s_{\pi(1)}, \dots, s_{\pi(\mu)}).$$

Let $(c_1, \dots, c_\mu) = H(z_{\pi(1)}, \dots, z_{\pi(\mu)}, m)$ be the challenge. We show that

$$s_{\pi(i)} \star \mathbf{pk}_{c_i} = z_{\pi(i)} \text{ for all } i \in [\mu].$$

We have $s_{\pi(i)} = r_{\pi(i)} \cdot s'_{\pi(i)} = r_{\pi(i)} \cdot h_{\pi(i)} \cdot g_{c'_{\pi(i)}}^{-1}$ and $c'_{\pi(i)} = c_{\pi^{-1}(\pi(i))} = c_i$. Therefore

$$\begin{aligned} s_{\pi(i)} \star \mathbf{pk}_{c_i} &= (r_{\pi(i)} \cdot h_{\pi(i)} \cdot g_{c_i}^{-1}) \star \mathbf{pk}_{c_i} = (r_{\pi(i)} \cdot h_{\pi(i)}) \cdot (g_{c_i}^{-1} \star \mathbf{pk}_{c_i}) \\ &= (r_{\pi(i)} \cdot h_{\pi(i)}) \star x = z_{\pi(i)}. \end{aligned}$$

E.2 Sequential one-more unforgeability

Theorem 13. *Let BS be Framework 2. For any p.p.t. algebraic adversary $\mathcal{A}$ we have that $\text{Adv}_{\mathcal{A}, \ell}^{\text{SOMUF}}(\lambda)$ is negligible. More precisely, for any such $\mathcal{A}$, that makes at most $Q = Q(\lambda)$ queries, there exists a p.p.t. adversary $\mathcal{B}$ and a negligible function ε such that*

$$\text{Adv}_{\mathcal{A}, \ell}^{\text{SOMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + Q\varepsilon + \binom{Q}{2}\varepsilon,$$

where $\varepsilon = \left(\frac{\mu}{\alpha, \dots, \alpha}\right)^{-1} = \frac{\alpha!^\kappa}{\mu!}$.

Proof. Before starting with the proof let us fix some notation. Let $\mathcal{A}$ be an adversary as in the statement. We assume that $\mathcal{A}$ makes, at most, a polynomial number of queries $Q = Q(\lambda)$ to the random oracle H . We denote by $(m_1, \mathbf{sig}_1), \dots, (m_{\ell+1}, \mathbf{sig}_{\ell+1})$ its output, on input the public key $\mathbf{pk} = (\mathbf{pk}_0, \dots, \mathbf{pk}_\kappa)$. More precisely, for $j \in [\ell + 1]$:

$$\mathbf{sig}_j = ((z_{1,j}, \dots, z_{\mu,j}), (s_{1,j}, \dots, s_{\mu,j})).$$

Since the adversary is algebraic, it will also provide a description of each set element $z_{i,j}$ in terms of the previously seen set element. That is, for all i, j it will give us $\alpha_{i,j} \in G$ and $x_{i,j} \in X$, where $x_{i,j}$ is a set element that $\mathcal{A}$ saw during its execution, such that $z_{i,j} = \alpha_{i,j} \star x_{i,j}$. We will also assume that all the forgeries output by $\mathcal{A}$ are valid signatures, that is for all i, j

$$s_{i,j} \star \mathbf{pk}_{c_{i,j}} = z_{i,j} \tag{11}$$

where for $c_{i,j}$ is defined as follows:

$$(c_{1,j}, \dots, c_{\mu,j}) = H(z_{1,j}, \dots, z_{\mu,j}, m_j).$$

For $J \in [\ell + 1]$ we define A_J and B_J as follows. A_J is the set of commitments $z_{i,J}$ that are simulated, that is the triplet $(z_{i,J}, c_{i,J}, s_{i,J})$ is a simulated transcript of the underlined sigma protocol. The set A_J is well defined. Since the adversary is algebraic, we say $z_{i,J}$ is *simulated* if and only if $z_{i,J} = s_{i,J} \star \text{pk}_{c_{i,J}}$. We define B_J as the set of commitments $z_{i,J} = \alpha_{i,J} \star x_{i,J}$ where $x_{i,j}$ is a signer commitment from a closed session (at the moment of the corresponding call to H) with the same challenge public key: namely, $c_{i,J}$ coincide with the challenge asked to the signer for $x_{i,J}$. We define $X_J = A_J \cup B_J$.

For $J \in [\ell + 1]$ let us denote with Open_J the set of indices corresponding to the sessions that are open when $\mathcal{A}$ queries the J -th forgery. We note that, by hypothesis on $\mathcal{A}$, $|\text{Open}_J| \leq 1$. For $I \in \text{Open}_J$ we define the set Ω_J^I as the collection of commitments in sig_J of the form $h \star y$ where y is a signer's commitment of session I , whose corresponding challenge matches the corresponding signer's response of session I (they are both with respect to the same public key).

Now we consider the following hybrid.

Game G_0 : The sequential one more unforgeability experiment $\text{sOMUF}(1^\lambda)$.

Game G_1 : This is G_0 with the following modification. When the adversary $\mathcal{A}$ outputs the signatures $(m_1, \text{sig}_1), \dots, (m_{\ell+1}, \text{sig}_{\ell+1})$, if there is a J such that $\mathcal{A}$ never queries $H((z_{i,J}, m_J))$ we emulate a query via lazy sample. Then, we abort the experiment if at least one of the following two conditions is satisfied.

1. There exists $J \in [\ell + 1]$ such that $|X_J| = \mu$.
2. There exist $J_1, J_2 \in [\ell + 1]$ and $I \in [\ell]$ such that $|X_{J_1} \cup \Omega_{J_1}^I| = |X_{J_2} \cup \Omega_{J_2}^I| = \mu$.

Lemma 8. *Let E be the event “ $\mathcal{A}$ aborts in G_1 ”. Then $\Pr[E]$ is negligible.*

Proof. For a positive integer $T \leq \mu$ we denote with p_T the probability that for a given commitment $\vec{\text{com}}$, message m and challenges $c_1, \dots, c_T$ we have $H(\vec{\text{com}}, m) = (c_1, \dots, c_T, *, \dots, *)$. Let E_1, E_2 be the events corresponding to the above abort conditions.

We have $\Pr[E] \leq \Pr[E_1] + \Pr[E_2]$. Let us first evaluate $\Pr[E_1]$. For $J \in [\ell + 1]$, we denote with Q_J the number of queries to H between the $(J - 1)$ -th and J -th forgery. We have that

$$\Pr[E_1] \leq \sum_{J \in [\ell + 1]} \Pr[|X_J| = \mu] = \sum_{J \in [\ell + 1]} Q_J \cdot p_\mu = Q \cdot \binom{\mu}{\alpha, \dots, \alpha}^{-1}$$

which is negligible by Lemma 4.

Now let us evaluate $\Pr[E_2]$. For $I \in [\ell]$ let Q_I be the number of queries to H made during session I . Let us denote with T_J the cardinality of X_J . We have

that:

$$\begin{aligned}
\Pr[E_2] &\leq \sum_{I \in [\ell]} \Pr[\exists J_1, J_2 \mid |X_{J_1} \cup \Omega_{J_1}^I| = |X_{J_2} \cup \Omega_{J_2}^I| = \mu] \\
&\leq \sum_{I \in [\ell]} \binom{Q_I}{2} \max_{T_{J_1}} \{p_{T_{J_1}} \cdot p_{\mu - T_{J_1}}\} \leq \sum_{I \in [\ell]} \binom{Q_I}{2} p_\mu \\
&\leq \binom{Q}{2} p_\mu = \binom{Q}{2} \binom{\mu}{\alpha, \dots, \alpha}^{-1}
\end{aligned}$$

which again is negligible by Lemma 4. We conclude that

$$\Pr[E] \leq \Pr[E_1] + \Pr[E_2] \leq (Q + \binom{Q}{2}) \binom{\mu}{\alpha, \dots, \alpha}^{-1}$$

is negligible.

We now proceed with the reduction to the *one-more vectorization problem*. Let $\mathcal{A}$ be a p.p.t. adversary playing G_1 . We construct an adversary $\mathcal{B}$ playing $G = \text{OMVP}$ as follows. Let $(x, X, G, \star)$ be the group action given in G .

1. Set $Y \leftarrow \text{Chall}()$.
2. Sample a random index $i^* \leftarrow [\kappa] \setminus \{1\}$. Set $g_1 = id$. For $i = 2, \dots, \kappa$ do the following. If $i = i^*$ set $\text{pk}_i = Y$, otherwise sample a group element $g_i \leftarrow G$ and set $\text{pk}_i = g_i \star x$.
3. Run $\mathcal{A}$ on input $(x, X, G, \star)$ and $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$.
4. Simulate the queries to H in the standard way and the queries to $\text{Sign}_1()$ and $\text{Sign}_2()$ as follows: let j denote the corresponding session,
 - Sign_1 : for $i \in [\mu]$ set $y_{i,j} \leftarrow \text{Chall}()$ and respond with $(y_{1,j}, \dots, y_{\mu,j})$.
 - Sign_2 : on input the challenge $(c_{1,j}, \dots, c_{\mu,j})$, for $i \in [\mu]$ set $s'_{i,j} \leftarrow \text{dLog}(\text{pk}_{c_{i,j}}, y_{i,j})$ and send $(s'_{1,j}, \dots, s'_{\mu,j})$. This simulation is correct since, $s'_{i,j} \star \text{pk}_{c_{i,j}} = y_{i,j}$ for all $i \in [\mu]$.
5. We receive $\ell + 1$ forgeries from $\mathcal{A}$

$$\begin{aligned}
m_1, \text{sig}_1 &= ((z_{1,1}, \dots, z_{\mu,1}), (s_{1,1}, \dots, s_{\mu,1})) \\
&\vdots \\
m_{\ell+1}, \text{sig}_{\ell+1} &= ((z_{1,\ell+1}, \dots, z_{\mu,\ell+1}), (s_{1,\ell+1}, \dots, s_{\mu,\ell+1}))
\end{aligned}$$

together with $\alpha_{i,j} \in G$ and $x_{i,j} \in X$ such that $z_{i,j} = \alpha_{i,j} \star x_{i,j}$, where $x_{i,j}$ is a set element saw during its run.

6. Our goal is to extract, from the forgeries, $h \in G$ such that $h \star x = Y$. Depending on the queries made by $\mathcal{A}$ to H we distinguish two cases.
Case 1. There exists $J \in [\ell + 1]$ such that $\mathcal{A}$ never queries $H((z_{i,j}), m_J)$. Then, there exists $I \in [\mu]$ such that $z_{I,J} \notin X_J$ (since $|X_J| < \mu$ by the first abort condition). If $z_{I,J} = \alpha_{I,J} \star x$ then, by Equation 11, we have

$$s_{I,J} \star \text{pk}_{c_{I,J}} = z_{I,J} = \alpha_{I,J} \star x,$$

Therefore, if $c_{I,J} = i^*$, we have $s_{I,J} \star Y = \alpha_{I,J} \star x$ and hence we have $h = s_{I,J}^{-1} \cdot \alpha_{I,J}$. Since H is a random oracle, $\Pr[c_{I,J} = i^*] = 1/\kappa$.

If there exists $i \in [\mu]$ and $j \in [\ell]$ such $z_{I,J} = \alpha_{I,J} \star y_{i,j}$ and $c_{i,j} \neq c_{I,J}$ or (because $z_{I,J} \notin X_J$). On one hand we have $y_{i,j} = s_{i,j} \star \text{pk}_{c_{i,j}}$, on the other, applying Equation 11, $y_{i,j} = \alpha_{I,J}^{-1} \star z_{I,J} = (\alpha_{I,J}^{-1} \cdot s_{I,J}) \star \text{pk}_{c_{I,J}}$. We thus obtain the relation

$$s_{i,j} \star \text{pk}_{c_{i,j}} = (\alpha_{I,J}^{-1} \cdot s_{I,J}) \star \text{pk}_{c_{I,J}}$$

with $c_{i,j} \neq c_{I,J}$. If $c_{i,j} = i^*$ we are able to extract h . Since H is a random oracle $\Pr[c_{i,j} = i^*] = 1/\kappa$. Otherwise, $z_{I,J} = \alpha_{I,J} \star \text{pk}_c$ for some c . Then $c_{I,J} \neq c$ (because the commitment is not simulated), and we can extract similarly.

Case 2. For all $J \in [\ell+1]$ the adversary queries $H((z_{i,J}), m_J)$. We distinguish two subcases.

- Exists a query that was made in between two sessions, that means before a i -th Sign_1 query and after the $(i-1)$ -th Sign_2 call. This case is analogous to the previous one and the probability of extraction is also $1/\kappa$.
- All the queries for the forgeries are done between some i -th Sign_1 and i -th Sign_2 call. It follows that there exists $I \in [\ell]$ and $J \in [\ell+1]$ such that $\mathcal{A}$ asks $H((z_{i,J}), m_J)$ and $H((z_{i,J+1}), m_{J+1})$ between the I -th Sign_1 and the I -th Sign_2 . Because we are in a sequential setting, all the i -th sessions for $i < I$ are closed when $\mathcal{A}$ queries for those forgeries. By the second abort condition in G_1 , there exists a commitment z in those forgeries of the form $z = \alpha \star x$ or $z = \alpha \star y$ for some y from the i -th Sign_1 commitments for $i \leq I$ and with different challenges. Then we can extract h with probability $1/\kappa$, as in case 1.

We have shown that from the forgeries returned by $\mathcal{A}$ we can extract h such that $Y = h \star x$ with non-negligible probability $1/\kappa$. It is easy to see that the knowledge of h allows us to recover all the other challenges $h_{i,j}$ such that $y_{i,j} = h_{i,j} \star x$. Therefore, up to re-run $\mathcal{A}$ a polynomial number of times, we let $\mathcal{B}$ respond to OMVP challenger with $(h, (h_{i,j}))$. It follows that

$$\text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) = \text{Adv}_{\mathcal{A}}^{G_1}(\lambda).$$

In conclusion,

$$\text{Adv}_{\mathcal{A},\ell}^{\text{sOMUF}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + \Pr[E] \leq \text{Adv}_{\mathcal{B}}^{\text{OMVP}}(\lambda) + Q\varepsilon + \binom{Q}{2}\varepsilon$$

where $\varepsilon = \left(\frac{\mu}{\alpha, \dots, \alpha}\right)^{-1}$.

F Calculation for Table 1

We detail how the data in Table 1 instantiated with CSIDH and LESS in terms of κ and μ . Denoting by grpSz the size (in kilobytes) of a group element, and by

setSz the size of a set element the data in Table 1 for Frameworks 1 and 2 are computed in function of grpSz, setSz, κ , and μ , as follows:

$$\begin{aligned} |\text{sk}| &= (\kappa - 1) \cdot \text{grpSz}, & |\text{pk}| &= (\kappa - 1) \cdot \text{setSz} \\ |\text{sig}| &= \mu \cdot (\text{grpSz} + \log_2(\kappa)/8000) \end{aligned}$$

The same formulas apply to Framework 3, except for the sizes are:

$$|\text{sig}| = \mu \cdot (\text{grpSz} + \log_2(\kappa)/8000) + \kappa \cdot \text{grpSz} + \kappa \log_2(\kappa)/8000,$$

where $\kappa \log_2(\kappa)$ is the cost of the representation of a permutation. Clearly, one can drop it by using a sorting algorithm to recover it.

We now discuss how the parameters κ and μ can be chosen. We will assume, from now on, that the security level is $\lambda = 128$ where the adversary is bound to $Q = 2^{80}$ random oracle queries. For Framework 2 (implying Framework 1 when $\kappa = 2$), the challenge space consists of strings having length μ over $[\kappa]$ and containing exactly μ/κ repetitions of each element of $[\kappa]$. Thus, to have a sufficiently large challenge space, it is enough to require that the multinomial coefficient satisfies

$$\binom{\mu}{\underbrace{\mu/\kappa, \dots, \mu/\kappa}_{\kappa \text{ times}}} \geq 2^{2\lambda}.$$

By Theorem 13, this condition is sufficient to enforce the (polynomially) sequential security of Framework 2. Some values for κ and μ satisfying this bound for $\lambda = 128$ are provided in Table 3. Note that, due to the fixed weight of the challenges, κ must divide μ .

κ	2 (Framework 1)	4	8	16	23	32	58
μ	262	136	96	80	69	64	58

Table 3. Some values of κ and μ for Scheme Framework 2, $\lambda = 128$.

This also shows that our previous choice of $\mu = 270$ for Framework 1 is enough to enforce its polynomially sequential security. Similarly, for Framework 3, one can choose parameters κ and μ such that Theorem 10 ensures concurrent security. For $\lambda = 128$, two valid choices for (κ, μ) are, for example, (70, 70) and (54, 108).

Sigma-protocol Line	
Scheme	Estimation
[70]	≈ 30 seconds (P42)
[65]	not mentioned
Fischlin-Line	
Scheme	Estimation
[3]	≈ 1 hour (P12)
[82]	not mentioned
[22]	If SHA-family hash + Generic ZKP(Ligero++): < 20 seconds (P3) If ZK-friendly hash + Lattice-based ZKP: < 1 second (P4)

Table 4. Known runtime information (estimation) in post-quantum blind signature literature.